



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Inteligencia Artificial

Predicción de la
degradación por ruido en
circuitos cuánticos NISQ
antes de su ejecución:
comparativa entre modelos
tabulares y redes
neuronales de grafos

Trabajo Fin de Estudios

Presentado por: Gregory Harutyunyan Gevorgyan

Dirigido por: Pablo Abellán Galiana

Ciudad: Granada

Fecha: 16 de agosto de 2026

Resumen

Los ordenadores cuánticos actuales devuelven resultados degradados por el ruido. Las técnicas habituales para corregirlo actúan después de medir y exigen ejecutar el circuito muchas veces, lo que multiplica el consumo de un recurso caro y escaso: el tiempo de procesador cuántico. Anticipar esa degradación permitiría decidir qué versión de un circuito merece la pena ejecutar, o si merece la pena ejecutarlo siquiera, sin gastar ese tiempo. Este trabajo estudia si la degradación puede predecirse antes de ejecutar, solo a partir de la estructura del circuito y de la calibración del procesador.

Para ello se construye un conjunto de veinte mil circuitos de cinco a quince qubits, etiquetados con su degradación exacta sobre la calibración real de un procesador de IBM durante cuarenta y dos días, y particionado en tres ejes de generalización independientes: tipo de circuito, tamaño y tiempo. Sobre él se comparan tres tipos de modelos bajo un mismo protocolo: una regresión Ridge, un Random Forest y una red neuronal de grafos.

El trabajo aporta tres resultados. Primero, la degradación con signo no es predecible antes de ejecutar y su reformulación como factor de supervivencia sí lo es. Segundo, Random Forest gana la comparativa, y la representación en grafo solo compensa al extrapolar a circuitos mayores o a algoritmos nunca vistos. Tercero, acertar sobre el factor no garantiza mitigar bien: Ridge lo predice con coeficiente positivo en los cinco observables y aun así empeora cuatro de ellos al aplicarlo.

Palabras clave: computación cuántica, NISQ, mitigación de errores, aprendizaje automático, redes neuronales de grafos.

Abstract

Today’s quantum computers return results degraded by noise. The usual techniques for correcting it act after measurement and require running the circuit many times, which multiplies the consumption of an expensive and scarce resource: quantum processor time. Anticipating that degradation would make it possible to decide which version of a circuit is worth running, or whether it is worth running at all, without spending that time. This work studies whether the degradation can be predicted before execution, using only the structure of the circuit and the calibration of the processor.

To that end, a dataset of twenty thousand circuits of five to fifteen qubits is built, labelled with their exact degradation over the real calibration of an IBM processor across forty-two days, and partitioned along three independent axes of generalisation: circuit type, size and time. Three types of model are then compared under a single protocol: a Ridge regression, a Random Forest and a graph neural network.

The work yields three results. First, the signed degradation is not predictable before execution, whereas its reformulation as a survival factor is. Second, Random Forest wins the comparison, and the graph representation only pays off when extrapolating to larger circuits or to previously unseen algorithms. Third, predicting the factor well does not guarantee good mitigation: Ridge predicts it with a positive coefficient across all five observables and yet worsens four of them when applied.

Keywords: quantum computing, NISQ, error mitigation, machine learning, graph neural networks.

Índice de Contenidos

Resumen	I
Abstract	II
1. Introducción	1
1.1. Motivación	1
1.2. Planteamiento del problema	3
1.2.1. Limitaciones de los enfoques actuales QEM	3
1.2.2. La pregunta de este trabajo	4
1.2.3. Propuesta	5
1.3. Estructura de la memoria	6
2. Contexto y Estado del Arte	7
2.1. Contexto del problema	7
2.1.1. Del bit al qubit	7
2.1.2. Qubits múltiples	9
2.1.3. Puertas cuánticas y circuitos	9
2.1.4. El circuito como grafo acíclico dirigido	11
2.1.5. El hardware actual	12
2.2. Estado del arte	14
2.3. Conclusiones	18
3. Objetivos y metodología de trabajo	19
3.1. Objetivo general	19
3.2. Objetivos específicos	20
3.3. Metodología de trabajo	20
4. Planteamiento de la comparativa	23
4.1. La variable objetivo	23
4.1.1. Los observables	23
4.1.2. El factor de supervivencia	25
4.1.3. El umbral de señal	28
4.1.4. Anatomía	30

4.1.5. Correlación	33
4.2. Creación del conjunto de datos	34
4.3. Análisis exploratorio de los datos	37
4.3.1. Observables sin etiqueta	38
4.3.2. Relación de cada familia de variables con la degradación	38
4.3.3. Telemetría de un mismo circuito y forma de los grafos	38
4.3.4. Colinealidad	40
4.3.5. Escalas, asimetrías y ángulos	40
4.4. Estrategia de entrenamiento	42
4.5. Métricas y validación	43
5. Desarrollo de la comparativa	45
5.1. Ridge	45
5.2. Random Forest	47
5.3. La red de grafos	48
6. Discusión y análisis de resultados	54
6.1. Resultados sobre el factor de supervivencia	54
6.1.1. Eje temporal: la deriva del hardware	55
6.1.2. Eje de tamaño: extrapolación a circuitos mayores	55
6.1.3. Eje de tipo: algoritmos nunca vistos	56
6.2. Resultados sobre la mitigación	58
7. Conclusiones y trabajo futuro	61
7.1. Conclusiones	61
7.2. Líneas de trabajo futuro	63
Referencias	65
A. Herramientas empleadas	69
B. Declaración de uso de herramientas de inteligencia artificial	70
B.1. Alcance de la declaración	70
B.2. Herramientas empleadas y uso concreto	70

Índice de Ilustraciones

1.1. Un procesador cuántico de IBM instalado en su criostato.	1
1.2. El esquema general de la mitigación por aprendizaje automático.	4
2.1. Un bit clásico frente a un qubit: dos estados posibles frente a estados infinitos sobre la esfera de Bloch.	8
2.2. Acción de la puerta de Hadamard sobre la esfera de Bloch.	10
2.3. Tres ejemplos de circuito: preparación de un estado GHZ, un ansatz <i>hardware-efficient</i> y una transformada cuántica de Fourier.	11
2.4. El grafo acíclico dirigido de un circuito de cinco qubits: cada puerta es un nodo y cada arista.	12
4.1. Los tres operadores de Pauli sobre la esfera de Bloch. Cada uno mide una dirección distinta del mismo estado, y los tres devuelven $+1$ o -1	24
4.2. Los cinco observables frente al ruido. Izquierda: dónde vive cada uno, con y sin ruido. Derecha: en cuántos circuitos el ruido lo encogió y en cuántos lo agrandó.	25
4.3. Las tres candidatas del objetivo, por observable y con tres umbrales de señal distintos. Random Forest, coeficiente de determinación en validación.	27
4.4. Pipeline de la propuesta de este trabajo	28
4.5. Lo que cuesta y lo que limpia cada umbral. Eje de abscisas en escala logarítmico.	29
4.6. Qué porcentaje de sus casillas pierde cada observable con el umbral aplicado.	30
4.7. La distribución del factor de supervivencia en los cinco observables.	31
4.8. Mediana del factor de supervivencia frente al número de qubits, en el conjunto de entrenamiento.	32
4.9. Correlación con signo entre los cinco objetivos, medida de dos formas. Cada panel lleva su propia escala de color; las dos van de -1 a $+1$	33
4.10. Recalibración diaria del procesador a lo largo de los 42 días.	35
4.11. El chip por dentro. A la izquierda, la región conexas sobre la que se coloca un circuito concreto. A la derecha, las averías y la invalidez del qubit 146.	36
4.12. Cómo se reparte el conjunto, y de dónde salen los tres ejes de generalización.	36

4.13. Cuánto predice cada familia de variables, por observable, medido de dos formas.	39
4.14. Cuánto varía la telemetría del qubit dentro de un mismo circuito.	39
4.15. Tamaño de los grafos del conjunto de entrenamiento. La línea marca la mediana.	40
4.16. Correlación entre las 54 variables candidatas, antes de descartar nada. . . .	41
4.17. Los seis peldaños de la estrategia de entrenamiento.	42
5.1. Correlación entre las 34 variables que quedan tras el descarte.	46
5.2. Los doce coeficientes mayores de Ridge, con su signo, promediando los cinco observables.	47
5.3. Las doce variables más importantes para Random Forest, promediando los cinco observables.	49
5.4. La arquitectura. El resumen global lee de todos los nodos y les devuelve: son las dos flechas, no una.	51
5.5. Pérdida de entrenamiento y de validación, época a época, para las tres convoluciones.	53
6.1. Coeficiente de determinación sobre el factor, por día de calibración, sobre circuitos aleatorios de hasta once qubits.	55
6.2. Coeficiente de determinación sobre el factor, por número de qubits. La línea a trazos marca el último tamaño presente en el entrenamiento.	56
6.3. Coeficiente de determinación sobre el factor, por tipo de circuito. Las celdas que se salen del eje van marcadas.	57
6.4. Mejora relativa sobre no mitigar, por observable y modelo. El cero es exactamente no corregir.	58

Índice de Tablas

2.1. Puertas cuánticas de un qubit más habituales.	10
2.2. Familias de procesadores cuánticos de IBM y su número de qubits.	13
2.3. El estado del arte frente a los cuatro rasgos que definen esta propuesta. . .	18
3.1. Concreción de las fases de CRISP-DM en este trabajo.	21
4.1. Los cinco observables objetivo y qué mide cada uno.	24
4.2. Coeficiente de determinación en validación de las tres candidatas (umbral fijo 0,01). Cada celda es el recorrido sobre los cinco observables.	26
4.3. El barrido del umbral de señal, medido sobre el conjunto de entrenamiento.	29
4.4. El factor de supervivencia por observable, con el umbral de 0,002 aplicado.	30
4.5. Los siete tipos de circuito y la razón de que estén.	37
5.1. Filas con etiqueta por observable, en entrenamiento y en validación.	45
5.2. Constante de regularización elegida para cada observable.	47
5.3. Hiperparámetros elegidos para cada observable.	48
5.4. Las cinco dimensiones podadas del vector de nodo, y su motivo.	50
5.5. Las mejores configuraciones del barrido, y la peor, sobre SAGEConv	52
5.6. Las tres convoluciones que se prueban y la configuración con la que se entrenó cada una.	52
6.1. Coeficiente de determinación sobre el factor, en validación.	54
6.2. Mejora relativa del observable corregido frente a no corregir.	58
6.3. Coeficiente de determinación sobre el factor, por eje de generalización. . . .	59
6.4. Resumen de la comparativa: los ocho peldaños frente a las dos métricas, por observable, sobre el conjunto de prueba completo (sin distinción de eje). . .	60
B.1. Herramientas de inteligencia artificial empleadas y tareas en las que se han utilizado.	70

1. Introducción

En este capítulo se introduce el propósito de esta memoria. El apartado 1.1 explica la utilidad de un ordenador cuántico y cómo el ruido les perjudica. El apartado 1.2 plantea el problema concreto que este trabajo aborda: corregir el ruido exige ejecutar el circuito, y ejecutarlo es caro; y enuncia la propuesta. Finalmente, el apartado 1.3 describe la organización de la memoria.

1.1. Motivación

La computación cuántica es un campo de la informática y la ingeniería que aprovecha las cualidades propias de la mecánica cuántica para resolver problemas que quedan fuera del alcance incluso de los ordenadores clásicos más potentes (Schneider y Smalley, 2026). Y esta definición hay que leerla con cuidado porque para la mayoría de las tareas se espera que los ordenadores clásicos sigan siendo la mejor solución. La ventaja cuántica no es general, sino que está concentrada en un tipo muy concreto de problema.

Ese tipo de problema es aquel en el que muchas variables interactúan de forma complicada. Un ejemplo es modelar el comportamiento de los átomos de una molécula, complejo por las múltiples interacciones entre sus electrones (Schneider y Smalley, 2026). Un ordenador clásico necesita *calcular* el comportamiento de la molécula, mientras que un ordenador cuántico no necesita hacer los cálculos, porque puede **imitar** el sistema molecular.

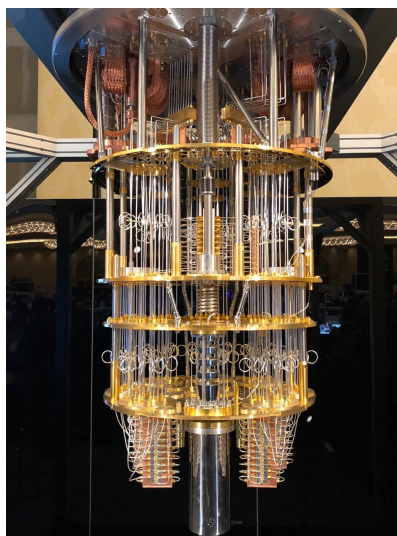


Figura 1.1: Un procesador cuántico de IBM instalado en su criostato.

Fuente: Clipset (2018).

Lo que se ve en la Figura 1.1 no es el ordenador cuántico, sino el frigorífico que lo contiene. El procesador cuántico es una pieza del tamaño de una moneda que va sujeta abajo del todo, a una temperatura cercana al cero absoluto. Toda esa maquinaria existe para un solo motivo: aislar los qubits del entorno, porque cualquier vibración o cualquier resto de calor los perturba. Esa fragilidad es justamente el problema del que trata esta memoria.

En general para hacerse una idea, los ordenadores cuánticos sirven, o se prevé que servirán, para solventar problemas complejos en campos como: industria farmacéutica, química y ciencia de materiales, energía, finanzas, biología computacional, aprendizaje automático, ciberseguridad y semiconductores (Schneider y Smalley, 2026). Históricamente, la primera aplicación práctica que se propuso fue el algoritmo de factorización de enteros publicado por Peter Shor en 1994 (Shor, 1994), que mostró cómo una máquina cuántica podría llegar a romper sistemas criptográficos.

Sin embargo, aunque la computación cuántica sea una tecnología tan prometedora, todavía presenta ciertos desafíos y limitaciones importantes.

Actualmente, todos los ordenadores cuánticos disponibles comercialmente (los de IBM, Google, IonQ. . .) pertenecen a una misma categoría, la llamada **era NISQ**, *Noisy Intermediate-Scale Quantum* (Ur Rehman, 2026). El físico John Preskill acuñó este término en 2018 para describir una generación de ordenadores cuánticos que ya son difíciles de simular con ordenadores clásicos, pero que siguen teniendo demasiados errores como para aprovechar todo el potencial en aplicaciones reales. (Ur Rehman, 2026).

Los qubits, la unidad básica de información en la computación cuántica, son extremadamente sensibles a cualquier perturbación de su entorno, y cada operación introduce una pequeña posibilidad de error. Estos errores se acumulan con el tiempo y limita la fiabilidad de los resultados obtenidos. Lo actuales sistemas disponen de entre decenas y unos pocos miles de qubits que no son suficientes para que los ordenadores cuánticos sean totalmente fiables. (Ur Rehman, 2026).

Aunque se conoce una posible solución, llamada Corrección de Errores Cuánticos (QEC, por sus siglas en inglés *Quantum Error Correction*), llevarla a la práctica sigue siendo uno de los mayores desafíos de la computación cuántica. Para hacerse una idea: para conseguir una buena fidelidad de un solo qubit es necesario ejecutar cientos o miles de ellos (Ur Rehman, 2026). Esto hace que los ordenadores cuánticos actuales estén aún lejos de poder aplicarse de manera eficaz.

Mientras tanto, existe una alternativa práctica en la que se centra nuestro trabajo: la Mitigación de Errores Cuánticos (QEM, por sus siglas en inglés *Quantum Error Correction*). En lugar de intentar eliminar el ruido, su objetivo es estimar cuál habría sido el resultado obtenido en un escenario ideal sin errores. Algunas de las técnicas que se incluyen en este área son: *Zero Noise Extrapolation* (ZNE), que estima el resultado ideal extrapolando las mediciones obtenidas de ruido; *Probabilistic Error Cancellation* (PEC), que compensa los efectos del ruido mediante una combinación probabilística; y *Readout Error Mitigation* (REM), que corrige los errores producidos durante la etapa de medición de los qubits..

1.2. Planteamiento del problema

1.2.1. Limitaciones de los enfoques actuales QEM

El problema de estos métodos no es la precisión, sino el **coste**. Tanto ZNE como PEC necesitan ejecutar el circuito **varias veces más** para poder estimar cuánto lo ha desviado el ruido. Como consecuencia, la corrección de errores necesita una cantidad adicional de tiempo de ejecución en el procesador cuántico, un recurso escaso y caro en la actualidad.

De ahí el interés, en este ámbito, del **aprendizaje automático** (ML-QEM, *Machine Learning - Quantum Error Mitigation*): si un modelo aprende la relación entre el ruido y su efecto, la corrección se calcula sin gastar ejecuciones adicionales. El procedimiento general consta de tres pasos (Liao y cols., 2024):

1. **Generación.** Se produce una gran cantidad de circuitos de entrenamiento que imitan la estructura del circuito de interés, de forma que el modelo aprenda sobre ejemplos parecidos a aquel que después tendrá que corregir.
2. **Codificación.** Cada circuito se pasa por un codificador que extrae sus características (el número de compuertas, su tipo o su profundidad, entre otras), convirtiendo el circuito en una representación que el modelo pueda consumir.
3. **Etiquetado.** Los circuitos se ejecutan en la QPU (Quantum Processing Unit) para obtener sus valores esperados con ruido. La comparación con el valor ideal conocido permite calcular la desviación introducida por el ruido, que se utiliza como variable objetivo del modelo.

Con ese conjunto de datos el modelo aprende a estimar la corrección, y una vez entrenado la aplica sin necesidad de las ejecuciones extra que hacen falta en ZNE o PEC. Ahora bien, el esquema tiene una limitación:

- el modelo toma como entrada el **valor ruidoso ya medido**, y por tanto es **necesario haber ejecutado el circuito** para poder corregirlo. La corrección sigue llegando después del gasto,
- y el **valor ideal** debe conocerse de antemano para entrenar el modelo.

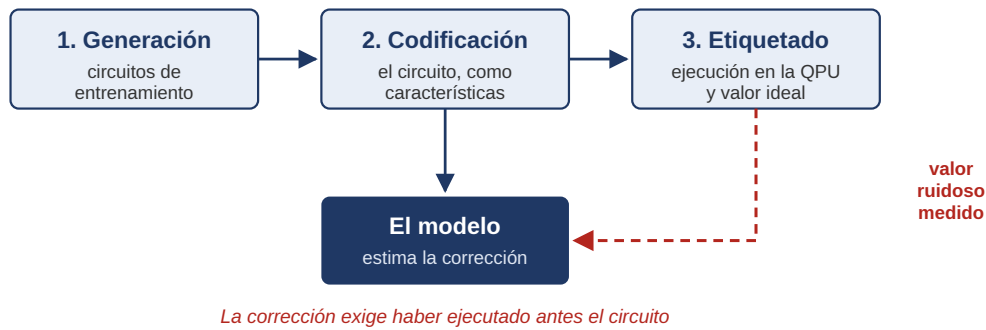


Figura 1.2: El esquema general de la mitigación por aprendizaje automático.

Fuente: Claude Code (Anthropic, 2026).

Para conseguir el valor ideal existen varios métodos pero se ejecutan en ordenadores clásicos. Como mostraremos más adelante, los ordenadores clásicos son incapaces de reproducir los procesos de los ordenadores cuánticos a partir de un número de qubits.

1.2.2. La pregunta de este trabajo

De ahí surge la pregunta que se trata en la memoria:

*¿Se puede predecir la degradación que sufrirá un circuito **antes de ejecutarlo**, mirando únicamente su estructura y la calibración del procesador?*

Lo que se propone no es cambiar el método, sino el momento en que se aplica la corrección (véase el Capítulo 4.1.2): en lugar de corregir el resultado una vez obtenido, intentar **anticiparlo**. Si esto fuera posible, su utilidad sería inmediata y diferente a la de los métodos anteriores. Permitirá estimar de antemano qué versión de un circuito merece

la pena ejecutar, o incluso si merece la pena ejecutarlo, sin consumir tiempo de procesador cuántico. Aunque la literatura ha explorado ideas similares, como clasificar circuitos según la calidad esperada de sus resultados antes de ejecutarlos (Hartnett, Barbosa, Mundada, Hush, y cols., 2024), todavía el enfoque ‘evitando recursos cuánticos’ sigue contando con escasas contribuciones.

1.2.3. Propuesta

Responder a esa pregunta exige, antes que un modelo, un conjunto de datos que no existe. De modo que lo primero que se propone es construirlo: **veinte mil circuitos** de entre cinco y quince qubits, simulados sobre el gemelo digital de un procesador de IBM y etiquetados con la degradación **exacta** que sufre cada uno. El ruido no se inventa, se reconstruye día a día a partir de la **calibración real** del procesador durante cuarenta y dos días. Y la partición se diseña para que el conjunto de prueba obligue a generalizar en **tres ejes independientes** (el tipo de circuito, su tamaño y el día de calibración).

Sobre esos datos se comparan **tres soluciones** bajo un mismo protocolo: una regresión lineal regularizada (**Ridge**), un **Random Forest** (el mejor modelo tabular de referencia según la ablación de Liao y cols. (2024)) y una **red neuronal de grafos con paso de mensajes** (MPNN, *message passing neural network*). La elección no es arbitraria, porque un circuito no es una fila: es una matriz de altura variable, una fila por puerta. Los dos primeros modelos exigen filas de longitud fija, así que hay que **resumir** el circuito en columnas, y ese resumen tira el orden y la estructura (dos circuitos con las mismas puertas estructuradas de formas distintas generan la misma fila). Sin embargo, la red de grafos consume el circuito tal cual. La diferencia entre unos y otra es, por tanto, **la medida de cuánto vale la estructura que el resumen descarta**. Este planteamiento está respaldado por trabajos previos. Por ejemplo, Tudisco, Volpe, Orlandi, y Turvani (2025) demuestra, antes de introducir su red basada en grafos, que características globales como el número de qubits, la profundidad del circuito o el recuento de puertas no son suficientes para distinguir adecuadamente las clases consideradas. Además, la comparación entre redes de grafos y Random Forest aplicada a circuitos cuánticos ya cuenta con un antecedente en Martyniuk, Jung, Barta, y Paschke (2025).

La última decisión de diseño que distingue esta propuesta de los enfoques habituales en la literatura es que los modelos no intentan predecir un único valor, sino cinco observables al mismo tiempo. La decisión de escoger varias en lugar de una se debe a que existen tipos

de circuitos donde varios observables valen cero y no hay nada que predecir. Ampliando el número de observables a cinco, conseguimos que todas las familias de circuitos aporten señales para el entrenamiento del modelo.

1.3. Estructura de la memoria

La memoria se organiza en **siete capítulos** y dos anexos:

1. **Introducción.** Presenta para qué sirve un ordenador cuántico y por qué el ruido limita hoy lo que puede resolver, delimita el problema concreto que se aborda y enuncia la propuesta.
2. **Contexto y estado del arte.** Reúne lo imprescindible para leer el resto —del qubit a las puertas y al circuito como grafo acíclico dirigido— y revisa lo publicado, ordenado por cuándo actúa cada familia de métodos respecto a la ejecución. Cierra marcando el hueco que este trabajo ocupa.
3. **Objetivos y metodología de trabajo.** Fija el objetivo general y los cinco específicos, la métrica que decide entre modelos y el proceso con el que se llega a ellos.
4. **Planteamiento de la comparativa.** Es el capítulo de diseño. Establece qué se predice (los observables, el factor de supervivencia y el umbral de señal), de dónde salen los datos y cómo se parten en tres ejes de generalización, y con qué métricas se juzga.
5. **Desarrollo de la comparativa.** Ataca los tres modelos uno a uno: qué datos ve cada uno, qué preprocesado es exclusivamente suyo y con qué arquitectura e hiperparámetros se quedó.
6. **Discusión y análisis de resultados.** Presenta lo que salió, separando dos preguntas distintas: cuánto acierta cada modelo al predecir el factor, y cuánto mejora el observable una vez aplicado.
7. **Conclusiones y trabajo futuro.** Enlaza cada objetivo con su conclusión, declara las limitaciones del trabajo y propone las líneas que las cierran.

Los anexos recogen las herramientas empleadas y la declaración de uso de herramientas de inteligencia artificial.

2. Contexto y Estado del Arte

2.1. Contexto del problema

Salvo indicación expresa, toda la teoría de esta sección está inspirada del manual de referencia de la disciplina, *Quantum Computation and Quantum Information* de Nielsen y Chuang (2010).

2.1.1. Del bit al qubit

El bit es el concepto fundamental de la computación y la información clásicas. La computación y la información cuánticas se construyen sobre un concepto análogo, el bit cuántico o, abreviadamente, el **qubit**.

Igual que un bit clásico tiene un estado (o 0, o 1), un qubit también tiene un estado. Dos estados posibles para un qubit son $|0\rangle$ y $|1\rangle$, que, como cabe suponer, se corresponden con los estados 0 y 1 de un bit clásico. La notación $|\rangle$ se denomina *notación de Dirac* y es la notación estándar para los estados en mecánica cuántica. La diferencia entre bits y qubits es que un qubit puede encontrarse en un estado distinto de $|0\rangle$ o $|1\rangle$: también es posible formar combinaciones lineales de estados, llamadas habitualmente **superposiciones**:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle. \quad (2.1)$$

Los números α y β son números complejos.

Un bit se puede examinar para determinar si está en el estado 0 o en el 1. Los ordenadores lo hacen constantemente, por ejemplo cuando recuperan el contenido de su memoria. Cabe recalcar que un qubit no se pueda examinar para determinar su estado cuántico, es decir, los valores de α y β . En su lugar, la mecánica cuántica nos dice que solo podemos adquirir una información mucho más restringida sobre el estado cuántico. Al medir un qubit obtenemos o bien el resultado 0, con probabilidad $|\alpha|^2$, o bien el resultado 1, con probabilidad $|\beta|^2$. Naturalmente, $|\alpha|^2 + |\beta|^2 = 1$, puesto que las probabilidades deben sumar uno.

Que un qubit pueda estar en un estado de superposición choca con nuestra comprensión de «sentido común» del mundo físico que nos rodea. Un bit clásico es como una moneda: o cara, o cruz. En monedas imperfectas puede haber estados intermedios, como quedarse

en equilibrio sobre el canto, pero en el caso ideal se pueden descartar. Un qubit, por el contrario, puede existir en un continuo de estados entre $|0\rangle$ y $|1\rangle$, hasta que se lo observa.

Los qubits no son una idea abstracta. Se han comprobado en muchos experimentos y existen distintas formas de construirlos en el mundo real:

- un qubit puede ser la polarización de un fotón, esto es, dos formas distintas de vibrar la luz;
- puede ser la orientación del espín de un núcleo dentro de un campo magnético;
- puede ser dos estados de un electrón en un átomo: el estado «fundamental» ($|0\rangle$) y el estado «excitado» ($|1\rangle$).

En este último ejemplo, el átomo funciona como un pequeño sistema de dos niveles. Si se le ilumina con luz de la energía adecuada, se puede hacer que el electrón pase de $|0\rangle$ a $|1\rangle$, o de $|1\rangle$ a $|0\rangle$. Hasta aquí, todo es clásico: dos estados, uno u otro.

Pero lo interesante llega después. Si no se le da suficiente tiempo de luz, el electrón no llega completamente a $|1\rangle$, sino que se queda en un estado intermedio, una superposición entre $|0\rangle$ y $|1\rangle$. Ese estado intermedio es lo que se denomina $|+\rangle$.

Un qubit se puede definir geométricamente sobre una esfera tridimensional. Esta esfera se denomina habitualmente **esfera de Bloch**, y nos permite visualizar el estado de un solo qubit. Se adjunta la Figura 2.1 para tener una idea visual del concepto del qubit.

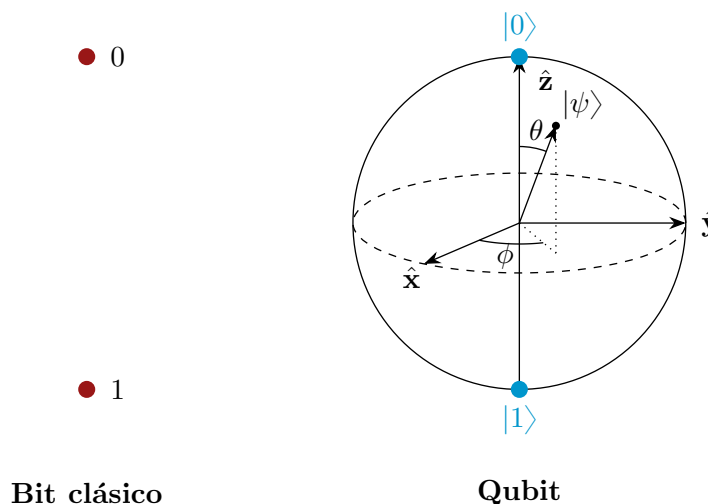


Figura 2.1: Un bit clásico frente a un qubit: dos estados posibles frente a estados infinitos sobre la esfera de Bloch.

Fuente: Claude Code (Anthropic, 2026).

2.1.2. Qubits múltiples

Un solo qubit no basta para calcular nada interesante. Entonces, ¿cómo se describe el estado conjunto de varios? La respuesta explica por sí sola por qué un ordenador cuántico puede hacer cosas que uno clásico no.

Con **dos qubits** las combinaciones posibles de valores clásicos son cuatro: 00, 01, 10 y 11. A cada una le corresponde un estado de la base, y el estado conjunto es una superposición de los cuatro, con una amplitud compleja por cada uno:

$$|\psi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle \quad (2.2)$$

Con **tres qubits** el razonamiento se repite. Las cadenas posibles pasan a ser ocho (de 000 a 111), y el estado necesita ocho amplitudes:

$$|\psi\rangle = \alpha_{000} |000\rangle + \alpha_{001} |001\rangle + \dots + \alpha_{111} |111\rangle \equiv \begin{pmatrix} \alpha_{000} & \alpha_{001} & \dots & \alpha_{111} \end{pmatrix}^T. \quad (2.3)$$

Y así sucesivamente:

- con **cuatro qubits**, los estados de la base van de $|0000\rangle$ a $|1111\rangle$ y hacen falta 16 **amplitudes**;
- con **cinco qubits**, van de $|00000\rangle$ a $|11111\rangle$ y hacen falta 32 **amplitudes**;
- ...

El patrón está claro: n **qubits requieren 2^n amplitudes complejas**. Con 50 qubits son más de mil billones de números, y describir ese estado en un ordenador clásico es imposible. Ahí está la promesa de la computación cuántica.

2.1.3. Puertas cuánticas y circuitos

Los cambios que experimenta un estado cuántico se pueden describir con el lenguaje de la computación cuántica. Del mismo modo que un ordenador clásico se construye a partir de un circuito eléctrico formado por cables y puertas lógicas, un ordenador cuántico se construye a partir de un circuito cuántico formado por cables y puertas cuánticas elementales, encargadas de transportar y manipular la información cuántica.

Los circuitos de un ordenador clásico constan de cables y puertas lógicas. Los cables sirven para transportar la información a lo largo del circuito, mientras que las puertas

lógicas realizan manipulaciones sobre esa información y la convierten de una forma en otra.

Puerta	Matriz	Efecto
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	Intercambia $ 0\rangle$ y $ 1\rangle$. Es la negación cuántica.
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	Intercambia los estados e introduce además una fase.
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	Deja $ 0\rangle$ intacto y cambia el signo de $ 1\rangle$.
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	Puerta de Hadamard: crea superposición a partir de un estado de la base.
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$	Aplica una fase de $\pi/2$ al estado $ 1\rangle$.
T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$	Aplica una fase de $\pi/4$.
$R_z(\theta)$	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$	Rotación de fase de ángulo arbitrario en torno al eje z .

Tabla 2.1: Puertas cuánticas de un qubit más habituales.
Fuente: elaboración propia.

En la Figura 2.2 se puede observar geométricamente cómo afecta una puerta Hadamard a un qubit en la esfera de Bloch.

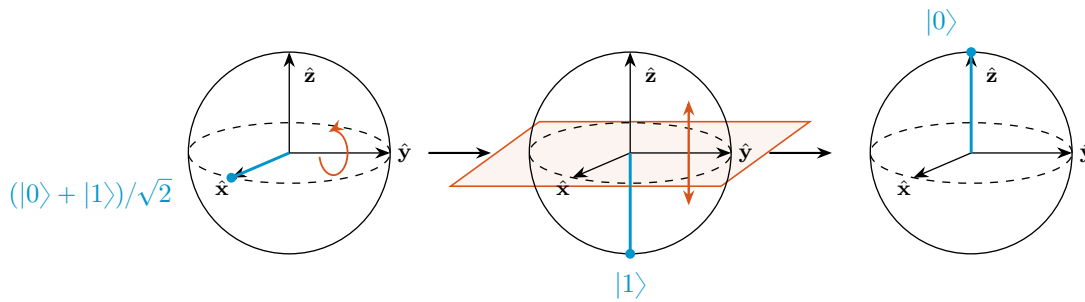
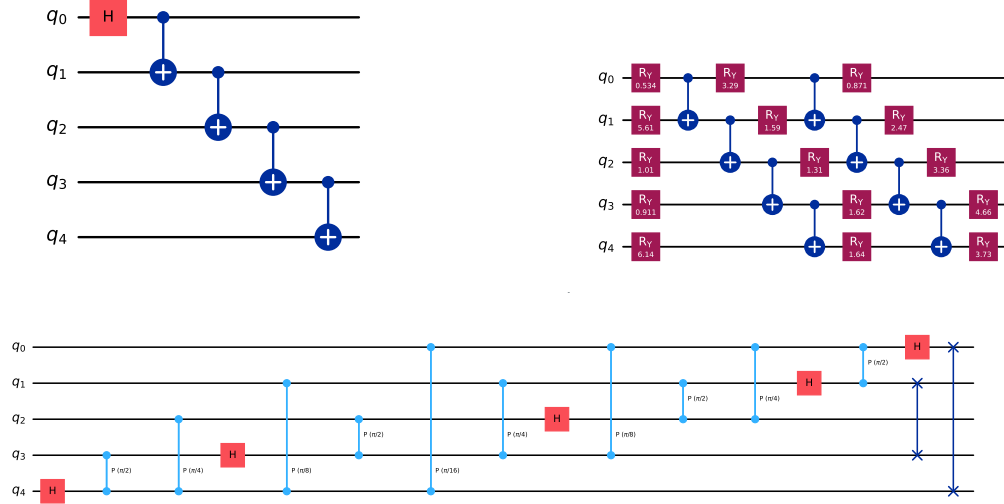


Figura 2.2: Acción de la puerta de Hadamard sobre la esfera de Bloch.
Fuente: Claude Code (Anthropic, 2026), a partir de Nielsen y Chuang, 2010.

El circuito. Un circuito cuántico es una secuencia de puertas aplicadas sobre un conjunto de qubits, seguida de una medición. Se representa con una línea horizontal por qubit (los «cables» del símil clásico) y las puertas colocadas sobre ellas en el orden en que se ejecutan. Al circuito lo describen dos magnitudes: la **anchura**, que es el número de qubits,

y la **profundidad**, que es el número de capas de puertas que deben ejecutarse una tras otra.

La Figura 2.3 recoge tres ejemplos, los tres de anchura 5, que sirven para ver hasta qué punto circuitos del mismo tamaño pueden ser muy distintos por dentro.



Fuente: elaboración propia.

Figura 2.3: Tres ejemplos de circuito: preparación de un estado GHZ, un ansatz *hardware-efficient* y una transformada cuántica de Fourier.

La base nativa. Un procesador real no implementa todas las puertas de la Tabla 2.1, sino un conjunto reducido a partir del cual puede construirse cualquier otra. Antes de ejecutarse, todo circuito se **traduce** a ese conjunto mediante un proceso llamado *transpilación*. En los procesadores IBM Heron la base nativa es $\{cz, id, rz, sx, x\}$.

Esa traducción evidencia más problema que aborda este trabajo. Un circuito teóricamente tiene diez puertas puede convertirse en cientos al expresarse en la base nativa, y cada puerta ejecutada es una oportunidad más de introducir error. Cuanto más largo es el circuito traducido, menos señal sobrevive hasta la medición.

2.1.4. El circuito como grafo acíclico dirigido

Un circuito se puede ver como un **grafo acíclico dirigido** (DAG, *directed acyclic graph*): cada puerta es un **nodo**, y hay una **arista** de una puerta a otra. Es acíclico porque los estados cuánticos evolucionan hacia adelante en el tiempo.

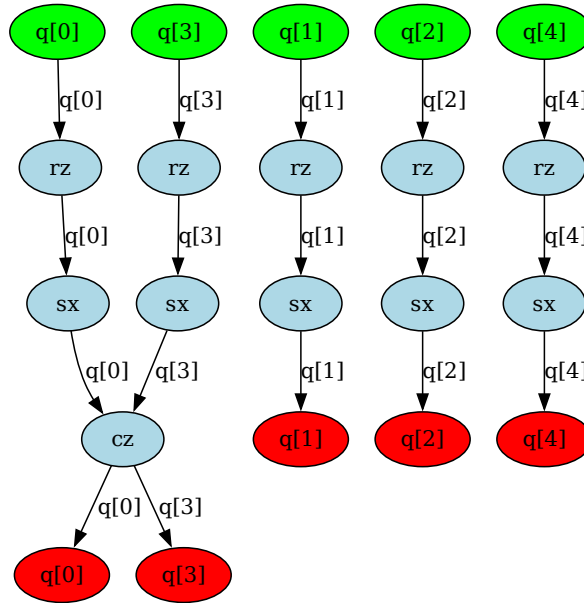


Figura 2.4: El grafo acíclico dirigido de un circuito de cinco qubits: cada puerta es un nodo y cada arista.

Fuente: elaboración propia.

Leído así, la profundidad del circuito deja de ser un recuento y pasa a ser el **camino más largo** del grafo. Es la magnitud que determina cuánto tiempo permanece vivo el estado cuántico y, por tanto, cuánto ruido acumula.

Y aquí es donde la física se convierte en un problema de aprendizaje automático: si un circuito es un grafo, predecir su comportamiento es un problema de regresión sobre grafos.

2.1.5. El hardware actual

Conviene hacerse una idea de qué son físicamente las máquinas de las que habla esta memoria, aunque solo sea para situar con una idea general el problema. Un procesador cuántico superconductor como los de IBM es un chip del tamaño de una moneda que vive en el fondo de un criostato: un cilindro de cobre dorado, de casi dos metros, cuya única función es mantener el chip a unas centésimas de grado por encima del cero absoluto. Los qubits no se programan escribiendo en una memoria, sino disparándoles pulsos de microondas por unos cables coaxiales; cada puerta del circuito es un pulso. Y la medición no devuelve un resultado, sino una estadística.

Al medir un qubit no se lee el estado en el que estaba: el estado **se destruye** y en su lugar aparece un cero o un uno, elegido al azar con la probabilidad que marcaba ese estado. Una única ejecución, por tanto, no informa de nada; devuelve una cadena de ceros y unos que podría haber salido de otra máquina completamente distinta. Lo que se hace en la práctica es ejecutar el mismo circuito miles de veces seguidas (cada repetición se llama *shot*) y anotar cuántas veces sale cada resultado. De ese recuento se obtiene el promedio que interesaba, y el circuito se ha ejecutado entero, desde cero, en cada una de esas miles de repeticiones. La consecuencia de todo esto es que la precisión es un factor que se paga caro, ya que para reducir el error en los resultados es necesario ejecutar más veces los circuitos, lo que aumenta coste de uso del procesador.

Merece de interés mencionar dos características del hardware que lo hacen distinto de un ordenador clásico:

Primera: los qubits no se hablan todos con todos. En un chip clásico cualquier dato puede llegar a cualquier registro; en uno cuántico cada qubit está conectado solo a dos o tres vecinos. Si un algoritmo necesita que interactúen dos qubits que no lo son, la única salida es ir acercando el estado de uno hasta el otro con puertas intermedias, y cada puerta añadida es ruido añadido.

Segunda: el chip no es el mismo dos días seguidos. Estas máquinas se recalibran a diario, y cada calibración publica un boletín con la tasa de error de cada puerta y los tiempos de vida de cada qubit. Esos números cambian y un qubit que ayer era fiable hoy puede no serlo, y hay puertas que se estropean y no se reparan. Esta es la razón de que el conjunto de datos del Capítulo 4 contenga la calibración real de un intervalo de días distintos en lugar de una única fotografía del chip.

Familia	Año	Qubits
Falcon	2019	27
Eagle	2021	127
Osprey	2022	433
Condor	2023	1 121
Heron	2023	133 / 156
Nighthawk	2025	120

Tabla 2.2: Familias de procesadores cuánticos de IBM y su número de qubits.

Fuente: elaboración propia a partir de IBM Quantum, 2026a.

Aunque 156 qubits parecen poca cosa habría que repasar **cuánta memoria haría falta en un ordenador normal para imitar el chip**. Y la respuesta es que describir

el estado de n qubits exige guardar 2^n números, de modo que **cada qubit que se añade duplica** el coste.

- Con 30 qubits hacen falta unos 17 GB, la memoria de un portátil actual bueno.
- Con 40 qubits, unos 17 TB, que ya es un servidor grande.
- Con 50 qubits, unos 18 PB: un centro de datos entero dedicado a imitar un chip del tamaño de una moneda.
- Y con los 156 qubits de un Heron sale un número de 48 cifras que ninguna máquina actual ni futura podrá almacenar.

Estas ejecuciones con ruido de chips cuánticos se pueden simular con ordenadores clásicos mediante paquetes de simuladores, como Qiskit Aer. Debido al gran cómputo que exigen una ejecución cuántica, los circuitos simulados en este trabajo llegan hasta 15 qubits.

Con todo esto hemos visto que el chip real maneja 156 qubits sin problema y el ordenador clásico no es capaz ni de acercarse. Por tanto, no se puede calcular de antemano cuánto va a degradar el ruido un resultado, porque ese cálculo es precisamente lo que resulta imposible. Lo que sí puede intentarse es **aprenderlo** a partir de ejemplos.

2.2. Estado del arte

Este apartado realiza un recorrido de lo que ya existe para corregir el efecto del ruido, ordenado por **cuándo** actúa cada familia de métodos respecto a la ejecución del circuito. Se ha escogido este criterio porque es el que separa a este trabajo de todo lo demás.

La mitigación clásica y su precio

La corrección de errores completa (QEC) exige cientos de qubits físicos por cada qubit lógico, inviable para las máquinas actuales (Preskill, 2018). La mitigación de errores (QEM) es la alternativa: no elimina el ruido, sino que estima el resultado que se habría obtenido sin él (Cai y cols., 2023).

Las técnicas de referencia son la **extrapolación a ruido cero** (ZNE), que trata de estimar cuál habría sido el resultado de un circuito si el ruido del hardware no estuviera presente (Temme, Bravyi, y Gambetta, 2017); la **cancelación probabilística de errores**

(PEC), que utiliza métodos estadísticos para compensar los errores introducidos durante la ejecución del circuito, aunque requiere un mayor número de ejecuciones (Cai y cols., 2023); y la **mitigación de lectura** (M3), centrada en corregir los errores que aparecen al medir el estado final de los qubits (Nation y Treinish, 2023).

Los tres planteamientos comparten los dos rasgos que este trabajo toma como punto de partida: todos **consumen ejecuciones adicionales** del procesador, que es el recurso más escaso, y todos actúan **después** de ejecutar.

Sustitución de las ejecuciones por un modelo

La primera forma de esquivar ese precio es reemplazar las ejecuciones extra por un modelo entrenado. Liao y cols. (2024) demostraron que una red puede aprender la corrección directamente a partir del valor ruidoso medido, y lo validaron sobre hardware real de IBM hasta un centenar de qubits. De su trabajo sale un dato que esta memoria aprovecha directamente: entre los modelos tabulares que comparan, **Random Forest es la mejor referencia**, y por eso es el que entra en la comparativa del Capítulo 5.

Sobre esa misma idea, pero llevándola al grafo, GTraQEM (Bao y cols., 2025) representa el circuito como un DAG (*Directed Acyclic Graph*) y lo procesa con un *graph transformer* apoyado en un **nodo virtual** conectado a todos los demás, que recoge el contexto global. Dos decisiones tuyas se reutilizan aquí y una se descarta. Se reutiliza el nodo virtual, por las razones que desarrolla el apartado 5.3. Se reutiliza también su formulación del objetivo como **residuo** (predicen la diferencia entre el valor ruidoso y el ideal, no el valor absoluto, porque esa diferencia es numéricamente pequeña y estabiliza el entrenamiento), que es el mismo razonamiento que conduce al factor de supervivencia del apartado 4.1.2. Y se descarta su arquitectura: un *transformer* sin paso de mensajes, cuyo coste de atención era superior a la máquina disponible. QEMFormer (Bao, Zhong, Ye, Tang, y Yan, 2025) se encuentra entre los métodos más avanzados y combina información general del circuito con detalles sobre su estructura.

Aquí aparece la diferencia que define esta memoria: los tres toman el **valor ruidoso medido como entrada**, de modo que hay que haber ejecutado el circuito para poder corregirlo. Placidi y cols. (2026) (preprint, sin actas confirmadas) miden cuánto pesa esa entrada, y concluyen que es la que más contribuye al rendimiento: sin ella, la precisión del modelo cae de forma considerable.

Técnicas pre-ejecución

Existe una familia distinta que decide **antes** de gastar la máquina. La herramienta oficial de IBM, mapomatic (Nation y Treinish, 2023), analiza distintas formas de asignar un mismo circuito al hardware disponible y selecciona la que previsiblemente producirá menos errores; es una heurística analítica, sin aprendizaje. Hartnett y cols. (2024) dan un paso más implementando técnicas de aprendizaje automático con datos medidos en hardware real.

Una nueva línea de investigación sí predice magnitudes sobre el grafo del circuito antes de ejecutarlo, y es lo más cercano a esta propuesta. Se destacan alguno de estos trabajos.

QuEst (Wang y cols., 2025) (preprint, sin actas confirmadas) estima la fiabilidad esperada de un circuito antes de ejecutarlo en la QPU. Construye el DAG, describe cada nodo con el tipo de puerta, el qubit sobre el que actúa y la telemetría de ese qubit, y lo procesa con un *graph transformer* cuya atención se restringe a los vecinos del grafo. Reportan un coeficiente de determinación de 0,991 en simulador ruidoso y por encima de 0,95 en cinco procesadores reales de IBM, con una predicción unas doscientas veces más rápida que simular. Sin embargo, los propios autores declaran que frente a una red tabular sobre variables agregadas, la ventaja media es de 0,014 de error cuadrático medio, pero **en tres de las cinco máquinas reales la diferencia baja de 0,0034 y en una de ellas gana la tabular**. Casi toda la ventaja se concentra en los circuitos estructurados.

Liu y colaboradores (Liu y cols., 2026), ya publicado en revista, predicen directamente el valor esperado que producirá el circuito, con un coeficiente de determinación entre 0,90 y 0,998 según el tamaño y la profundidad. La aportación más relevante es la siguiente: con el *mismo* modelo y la *misma* arquitectura, cambiar qué magnitud se predice les mejora el acierto en un 36 %. Es una evidencia clara de que **reformular el objetivo puede cambiar de modelo**, que es exactamente lo que mide el apartado 4.1.2 de esta memoria.

Tudisco y colaboradores (Tudisco y cols., 2025), en actas de IEEE QCE 2025, resuelven otro problema (en qué máquina conviene ejecutar) pero su análisis previo es lo que interesa: antes de proponer su red, **demuestran que el número de qubits, la profundidad y el recuento de puertas no bastan** para separar los casos. De ahí concluyen que hay que meter el circuito entero en el modelo en vez de resumirlo a mano. Es el mismo argumento que sostiene la comparativa de esta memoria.

Aktar y colaboradores (Aktar, Bärtschi, Oyen, Eidenbenz, y Badawy, 2024), también en actas de IEEE QCE, predicen la expresibilidad de circuitos parametrizados con una red de grafos adaptada de QuEst. Aportan dos medidas que este trabajo utiliza. La primera es la evidencia en la **extrapolación en tamaño**: un modelo entrenado hasta tres qubits se dispara a un error de 0,22 al predecir seis, y uno entrenado hasta ocho se queda en 0,05 al predecir diez. La segunda conclusión es que **La variedad de condiciones de ruido durante el entrenamiento es más importante que entrenar con el perfil de ruido exacto de la máquina objetivo..** Al evaluar sobre un dispositivo cuántico real, el modelo entrenado con tres perfiles de ruido diferentes casi duplica la tasa de acierto del modelo entrenado exclusivamente con el perfil de ruido de esa misma máquina. Este resultado aporta evidencia empírica a favor de entrenar con los cuarenta y dos días de calibración disponibles, ya que la variabilidad en las condiciones de ruido parece favorecer una mejor capacidad de generalización. De ellos sale además el precedente publicado de la **pérdida de Huber** que usa el apartado 5.3.

SQuASH (Martyniuk y cols., 2025) (preprint) es el **precedente directo de esta comparativa**: enfrenta una red de grafos a un Random Forest sobre los mismos circuitos y el mismo protocolo, y la red gana en las tres familias que prueban, con coeficientes de 0,945 frente a 0,705 en el caso más favorable.

Ninguno de los cinco predice la degradación de un observable, y ninguno modela el paso del tiempo.

El hardware cambia con el tiempo

Hirasaki, Daimon, Itoko, Kanazawa, y Saitoh (2023) midieron que las tasas de error no se degradan de forma progresiva, sino **a saltos bruscos** que aparecen en cualquier momento y duran minutos o decenas de minutos, y que la recalibración diaria de IBM no basta para capturarlos. Huo, Leeds, Ludmir, DiBrita, y Patel (2026), desde la perspectiva de sistemas, lo confirman por el otro extremo: el mismo programa ejecutado en momentos distintos devuelve resultados inconsistentes, y proponen una técnica que reduce esa variabilidad. Uno aporta la evidencia física del fenómeno y el otro su impacto medible sobre el usuario final. Entre los dos justifican que el conjunto de datos de esta memoria tenga un **eje temporal** con días de calibración reales.

Trabajo	Pre-ejec.	Magnitud	Multi-salida	Deriva
ZNE, PEC, M3	no	sí	no	no
Liao y cols., 2024	no	sí	no	parcial
GTraQEM, QEMFormer	no	sí	sí	no
mapomatic, Hartnett y cols., 2024	sí	no	no	no
Wang y cols., 2025	sí	sí	no	no
Liu y cols., 2026	sí	sí	no	no
Aktar y cols., 2024	sí	sí	no	no
Martyniuk y cols., 2025	sí	sí	no	no
Tudisco y cols., 2025	sí	no	no	no
Este trabajo	sí	sí	sí	sí

Tabla 2.3: El estado del arte frente a los cuatro rasgos que definen esta propuesta.
Fuente: elaboración propia.

2.3. Conclusiones

El recorrido del estado del arte deja ver que las investigaciones están partidas en dos mitades. Una corrige bien, pero solo **después** de haber gastado la máquina. La otra decide **antes**, pero lo que predice sirve para descartar un circuito o para elegir dónde ejecutarlo, no para corregir su resultado.

La Tabla 2.3 contrasta el corpus con los cuatro rasgos que definen esta propuesta, y **ninguna fila reúne los cuatro**:

1. **Pre-ejecución**: la entrada del modelo nunca incluye el resultado medido. Es la frontera entre las dos mitades, y la que decide si el método ahorra tiempo de procesador o lo consume.
2. **Magnitud**: predecir cuánto se degrada, y no solo si conviene ejecutar.
3. **Multi-salida** sobre varios observables a la vez, en lugar de un escalar único. Los trabajos pre-ejecución revisados predicen todos una sola cantidad.
4. **El paso del tiempo como eje**, con días de calibración reales y disjuntos entre entrenamiento y prueba. Hay evidencia publicada de que el hardware deriva a saltos, y aun así ningún trabajo de la literatura lo incluye en su evaluación.

3. Objetivos y metodología de trabajo

Para este trabajo se plantea un objetivo general, del que derivan objetivos específicos. Después de definir los objetivos, se explicará la metodología empleada para cumplirlos.

3.1. Objetivo general

El objetivo de este trabajo es realizar un **análisis comparativo de soluciones de tres técnicas de aprendizaje automático que predicen, antes de ejecutar un circuito cuántico y únicamente a partir de su estructura y de la calibración del procesador, la degradación que el ruido causará en sus resultados**. En dicho análisis se obtiene un conjunto de métricas estándar (R^2 , MAE y Mejora Relativa¹), a través de las que se determina cuál de las técnicas ofrece la mejor solución, así como si estas técnicas superan una determinada «línea base».

Esta definición cumple los criterios «SMART», ampliamente aceptados como buena práctica para la fijación de objetivos:

- **S: *Specific*** (específico): comparar tres técnicas de aprendizaje automático (regresión Ridge y Random Forest y tres variantes de GNN) para predecir el **factor de supervivencia** de los observables tras la degradación por ruido sin ejecutar el circuito. Se observa además si superan la línea base.
- **M: *Measurable*** (medible): la comparativa se realiza a través de un conjunto de métricas estándar (MAE, R^2 y Mejora Relativa), desglosadas por los tres ejes de generalización y por cada uno de los cinco observables. **La métrica de referencia que decide entre los modelos es el R^2 sobre el factor de supervivencia**, medido sobre el conjunto de prueba. Se toma esta, y no la **Mejora Relativa** sobre el resultado ya mitigado, porque esta última exige conocer el valor ruidoso (que solo existe después de ejecutar) y la utilidad del trabajo reside precisamente en la predicción pre-ejecución.

¹La Mejora Relativa se calcula como $1 - (\text{MAE}_{\text{mitigado}}/\text{MAE}_{\text{ruidoso}})$. Representa la fracción del error cuántico original que el modelo ha conseguido eliminar, donde un valor mayor que cero indica que la predicción supera a la línea base de no mitigar.

- **A: *Attainable*** (alcanzable): se apoya en el estado del arte desarrollado en el Capítulo 2.2, en un conjunto de datos (con tarea de generarlos) de veinte mil circuitos con calibración real, y en tres técnicas de modelado documentadas.
- **R: *Relevant*** (relevante): tal y como se ha expuesto en el Capítulo 1, poder anticipar la degradación permite decidir qué circuitos merece la pena ejecutar sin consumir tiempo de procesador cuántico, un recurso muy caro y escaso en la actualidad.
- **T: *Time-Related*** (con un tiempo determinado): se desarrolla dentro del marco temporal establecido para la realización del Trabajo Fin de Máster durante el curso académico.

3.2. Objetivos específicos

Los objetivos específicos de este trabajo son los siguientes:

1. **Análisis del estado del arte y contextualización del problema.**
2. **Construcción del conjunto de datos**, con calibración real del procesador y tres ejes de generalización independientes (tipo de circuito, tamaño y tiempo).
3. **Desarrollar diferentes estrategias de representación de la información**, es decir, cómo representar el factor de degradación y comprobar si realmente es predecible antes de ejecutar, midiendo el R^2 en validación sobre cada candidato.
4. **Entrenar y validar modelos** de Ridge, Random Forest y grafos neuronales mediante un protocolo experimental homogéneo, y determinar un posible modelo óptimo.
5. **Comparar el rendimiento** de los distintos enfoques utilizando métricas objetivas.

3.3. Metodología de trabajo

Se sigue una metodología **empírico-cuantitativa**. Tras haber planteado el problema y haber revisado la información que existe sobre este, se establece la hipótesis de la investigación, que puede formularse como sigue: «*al menos uno de los modelos de la comparativa es capaz de predecir la degradación por ruido de un circuito cuántico antes de ejecutarlo mejor que los demás y de superar la línea base establecida por el valor medio de la variable objetivo*». Este estudio trata de comprobar la validez de dicha hipótesis.

Fase de CRISP-DM	Concreción en este trabajo
Comprensión del negocio	Revisión del estado del arte, identificación del hueco y formulación de la hipótesis nula.
Comprensión de los datos	Análisis exploratorio de la calibración real del procesador y de las etiquetas generadas; detección de las magnitudes que no son predecibles.
Preparación de los datos	Generación del conjunto, definición de la partición temporal, agregación a variables tabulares y construcción de los grafos.
Modelado	Implementación y ajuste de los tres modelos con el mismo protocolo, buscando los hiperparámetros contra el conjunto de validación.
Evaluación	Métricas desglosadas por los tres ejes y por observable, y contraste frente a la línea base.
Despliegue	Publicación del código y de los artefactos, y documentación de los límites de la solución.

Tabla 3.1: Concreción de las fases de CRISP-DM en este trabajo.

Fuente: elaboración propia a partir de Wirth y Hipp, 2000.

El desarrollo se estructura sobre **CRISP-DM** (Wirth y Hipp, 2000), el modelo de proceso de referencia en proyectos de minería de datos, que sigue siendo el marco más empleado dos décadas después de su publicación (Schröer, Kruse, y Gómez, 2021). La Tabla 3.1 recoge cómo se concreta cada una de sus seis fases en este trabajo.

A modo de listado, para alcanzar los objetivos propuestos se siguen los siguientes **pasos**:

1. **Revisión del estado del arte** y de los trabajos relacionados.
2. **Determinación de los modelos** que forman parte de la comparativa, a partir del análisis del estado del arte.
3. **Diseño y generación del conjunto de datos**, con la calibración real del procesador y los tres ejes de generalización (tipo, tamaño y tiempo).
4. **Análisis exploratorio** de los datos y **validación de las etiquetas**, realizado únicamente sobre el conjunto de entrenamiento.
5. **Determinación de la variable objetivo**, descartando las magnitudes que se midan como no predecibles.
6. **Preparación de los datos**: partición temporal, agregación a variables tabulares y construcción de los grafos.
7. **Implementación** de los tres modelos en Python.

8. **Entrenamiento y ajuste** de los modelos. La búsqueda de hiperparámetros se realiza contra un conjunto de validación **temporal**, formado por los últimos días de calibración del entrenamiento, y no por validación cruzada aleatoria: una división al azar mezclaría días entre ambos conjuntos, inflando los resultados y ocultando cómo rendirá el modelo ante futuros datos.
9. **Obtención de las métricas** de los distintos modelos (R^2 , MAE y Mejora Relativa) desglosadas por eje de generalización y por observable.
10. **Validación de los resultados** y contraste frente a la línea base.
11. **Determinación de un posible modelo óptimo**, estrictamente mejor que el resto, o identificación de otras casuísticas.
12. **Elaboración de las conclusiones.**

4. Planteamiento de la comparativa

En esta sección se repasarán las variables objetivo, las razones de su elección y su anatomía. A continuación, se detallará la construcción del conjunto de datos y se realizará su correspondiente análisis exploratorio. Finalmente, se cierra la sección mostrando los pasos del preprocesado que se han llevado a cabo, previo al entrenamiento de los modelos.

4.1. La variable objetivo

En este capítulo se reproduce el razonamiento que se siguió para construir la variable objetivo, un proceso que supuso grandes desafíos. Se comienza introduciendo el concepto de los observables y por qué se eligieron esos cinco. Después se listan las tres candidatas a variable objetivo, se mide cuál de ellas es aprendible y se explica cómo se reconstruye la degradación sin renunciar a predecir antes de ejecutar. A continuación se justifica el umbral por debajo del cual una muestra deja de tener señal que aprender y se dedica un pocas páginas a describir la anatomía de la elección y las decisiones que se tomaron respecto a su física. El capítulo cierra con las limitaciones de las decisiones tomadas.

4.1.1. Los observables

Salvo que se indique lo contrario, la teoría de esta sección sale del mismo libro de cabecera del Capítulo 2.1: *Quantum Computation and Quantum Information*, de Nielsen y Chuang (2010).

Definición y elección

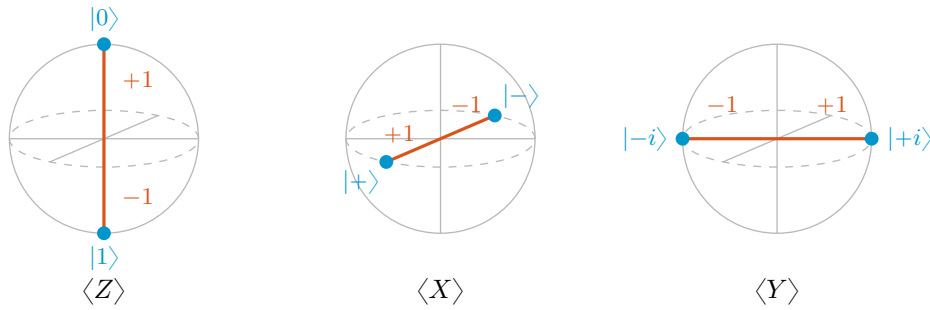
Un circuito acaba en un estado $|\psi\rangle$ que no se puede leer: al medir se obtiene una cadena de ceros y unos y el estado colapsa (apartado 2.1.1). La única forma de extraer información útil es repetir la ejecución y quedarse con un promedio. Lo que se promedia es un **observable**, una magnitud física medible representada por un operador O , y la cantidad que interesa es su **valor esperado**:

$$\langle O \rangle = \langle \psi | O | \psi \rangle = \psi^\dagger O \psi, \quad (4.1)$$

No es una probabilidad, sino **un número real**.

Los observables de este trabajo se construyen con los tres **operadores de Pauli**, X , Y y Z , que son las proyecciones elementales que admite un qubit, una por cada eje de la esfera de Bloch (Figura 4.1). Los tres tienen los mismos dos autovalores, $+1$ y -1 : sea cual sea el estado y sea cual sea el eje, una medida individual solo puede devolver uno de esos dos números. De ahí sale la propiedad siguiente: el valor esperado es un promedio de $+1$ y -1 y, por tanto, vive en $[-1, 1]$ por construcción.

Figura 4.1: Los tres operadores de Pauli sobre la esfera de Bloch. Cada uno mide una dirección distinta del mismo estado, y los tres devuelven $+1$ o -1 .



Fuente: Claude Code (Anthropic, 2026).

Con esas tres piezas se puede construir cualquier observable de Pauli. Los elegidos para este trabajo son los cinco que se muestran en la Tabla 4.1: las tres bases de Pauli promediadas sobre todos los qubits, y dos correladores, uno global y otro local. Con estos cinco cubrimos lo básico: el fallo en cúbits individuales y los errores que surgen al relacionarlos.

Tabla 4.1: Los cinco observables objetivo y qué mide cada uno.

Fuente: elaboración propia.

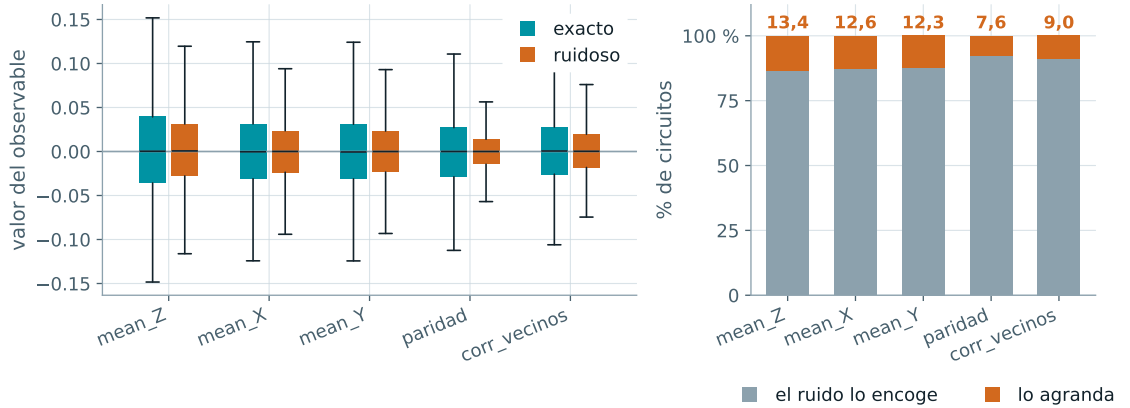
Observable	Definición	Qué mide
Media de Z	$\frac{1}{n} \sum_i \langle Z_i \rangle$	La magnetización media en la base de medida: cuánto se inclinan los qubits hacia $ 0\rangle$ o hacia $ 1\rangle$.
Media de X	$\frac{1}{n} \sum_i \langle X_i \rangle$	Lo mismo en la base de las superposiciones. Es independiente de Z .
Media de Y	$\frac{1}{n} \sum_i \langle Y_i \rangle$	Con este se consigue las tres direcciones independientes.
Paridad	$\langle Z_0 Z_1 \cdots Z_{n-1} \rangle$	Magnitud global : un solo operador sobre todos los qubits a la vez.
Correlador de vecinos	$\frac{1}{n-1} \sum_i \langle Z_i Z_{i+1} \rangle$	Correlación local .

Impacto del ruido a un valor esperado

Los cinco observables están acotados en todo el intervalo $[-1, 1]$, pero la mayor parte se acumula en un intervalo más reducido $[-0,5, 0,5]$, como se puede observar en el panel izquierdo de la Figura 4.2. Además, ese mismo panel sugiere que los valores exactos tras la ejecución se reducen, ya que los rangos usuales de los valores ruidosos son menores. Sin embargo, como se muestra en el panel derecho de la Figura 4.2, el ruido afecta de dos maneras:

- **Fenómeno de encogimiento:** se produce en gran parte de los circuitos (entre el 86,6 % y el 92,4 % de las veces).
- **Fenómeno de expansión:** se produce en menor número de circuitos (entre el 7,6 % y el 13,4 %).

Figura 4.2: Los cinco observables frente al ruido. Izquierda: dónde vive cada uno, con y sin ruido. Derecha: en cuántos circuitos el ruido lo encogió y en cuántos lo agrandó.



Fuente: elaboración propia.

Entonces el ruido puede amplificar y no es una anomalía de la simulación.

4.1.2. El factor de supervivencia

Candidatas a variable objetivo y elección

La pregunta del trabajo, *¿cuánto degrada el ruido?*, tuvo 3 opciones a poder responderlo:

1. **La degradación con signo**, que era el objetivo de inicial del proyecto:

$$\Delta = \langle O \rangle_{\text{exacto}} - \langle O \rangle_{\text{ruidoso}}. \quad (4.2)$$

Es lo que se quiere corregir y pedecirla permitiría sumarla al resultado medido.

2. **La magnitud de la degradación**, $|\Delta|$, que renuncia a la dirección del error y no sirve para corregir.
3. **El factor de supervivencia**, la fracción de señal que queda tras la ejecución:

$$f = \frac{\langle O \rangle_{\text{ruidoso}}}{\langle O \rangle_{\text{exacto}}}. \quad (4.3)$$

- $f = 1$, si el circuito salió intacto.
- $f < 1$, si el ruido redujo.
- $f > 1$, si el ruido amplificó.
- $f = 0$, destruyó la señal completamente.

La elección se midió. Se ajustó el mismo modelo, con las mismas variables y la misma partición, cambiando únicamente el objetivo, y se evaluó en validación: mismos tipos de circuito y mismo rango de tamaños que el entrenamiento. Se usaron los dos modelos tabulares de la comparativa y se repitió todo con tres umbrales de señal distintos (la explicación de estos umbrales se realiza en el Capítulo 4.1.3), para comprobar que la conclusión no dependía de esa constante (Tabla 4.2 y Figura 4.3).

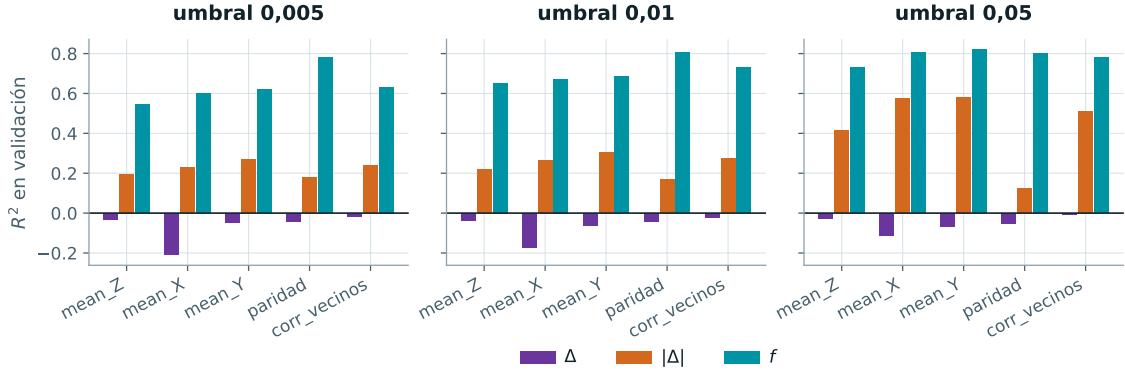
Tabla 4.2: Coeficiente de determinación en validación de las tres candidatas (umbral fijo 0,01). Cada celda es el recorrido sobre los cinco observables.

	Objetivo	Ridge	Random Forest
Fuente: elaboración propia.	Δ con signo	-0,019 ... +0,005	-0,175 ... -0,021
	$ \Delta $	+0,260 ... +0,366	+0,167 ... +0,303
	Factor de supervivencia	+0,657 ... +0,782	+0,652 ... +0,804
Umbral de señal 0,01.			

El resultado no admite matices de lectura:

- **La degradación con signo no se aprende.** Su coeficiente de determinación es nulo o negativo en los cinco observables, con los dos modelos y con los tres umbrales.
- **La magnitud sí se aprende, pero poco.** Renunciar al signo recupera señal predecible. Pero un objetivo sin signo no corrige nada.
- **El factor de supervivencia es el único aprendible de verdad**, y duplica a $|\Delta|$ en los dos modelos y en los tres umbrales.

Figura 4.3: Las tres candidatas del objetivo, por observable y con tres umbrales de señal distintos. Random Forest, coeficiente de determinación en validación.



Fuente: elaboración propia.

La reformulación

De las tres candidatas, la única que el modelo puede aprender es el factor de supervivencia, y esa es la que se escoge como objetivo de entrenamiento. La degradación se reconstruye después, a partir del factor predicho y del valor medido tras ejecución:

$$\hat{\Delta} = \langle O \rangle_{\text{ruidoso}} \cdot \frac{1 - \hat{f}}{\hat{f}}. \quad (4.4)$$

Una aclaración: para reconstruir la degradación, en gran parte de la literatura Placidi y cols. (2026) recurren al valor ruidoso $\langle O \rangle_{\text{ruidoso}}$ (obtenido tras ejecución en **hardware cuántico real** y costoso pero alcanzable) y del valor $\langle O \rangle_{\text{exacto}}$ (obtenido tras ejecución en **hardware clásico** y costoso pero con alcance limitado, explicado en el Capítulo 2.1.5)

Sin embargo, en nuestra propuesta nunca haría falta calcular el valor $\langle O \rangle_{\text{exacto}}$ para poder mitigar el ruido, donde existen actualmente limitaciones a partir de un cierto número de qubits.

A partir de la igualdad 4.4, obtenemos la reconstrucción del valor mitigado así:

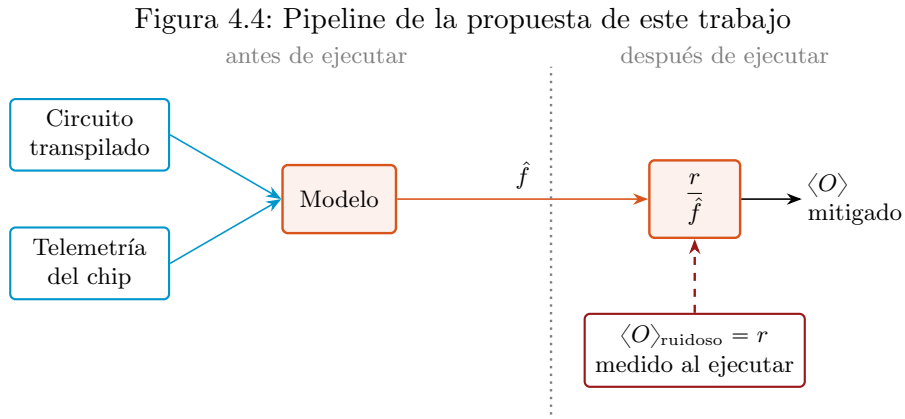
$$\langle O \rangle_{\text{mitigado}} = \langle O \rangle_{\text{ruidoso}} + \hat{\Delta} = \frac{\langle O \rangle_{\text{ruidoso}}}{\hat{f}}. \quad (4.5)$$

Conviene profundizar en la idea propuesta, ya que este planteamiento no rompe el **paradigma pre-ejecución**. Las razones son las siguientes:

- **La entrada del modelo** está compuesta únicamente por el circuito y la telemetría

del chip. El modelo **nunca ve** el resultado de la ejecución, ni al entrenar ni al predecir.

- **El valor medido entra solo tras la predicción** para mitigar el ruido si corresponde la decisión.
- **Y la predicción sirve sin ejecutar nada.** El factor predicho es, por sí solo, la respuesta a la pregunta del apartado 1.2.2: dice qué fracción de la señal sobrevivirá antes de gastar un ciclo de procesador. La reconstrucción es una aplicación posterior y opcional.



Fuente: Claude Code (Anthropic, 2026).

4.1.3. El umbral de señal

El factor de supervivencia es un cociente, y cuando el denominador se acerca a cero se dispara.

$$f = \frac{\langle O \rangle_{\text{ruidoso}}}{\langle O \rangle_{\text{exacto}}} \implies \lim_{|\langle O \rangle_{\text{exacto}}| \rightarrow 0} |f| = \infty \quad (4.6)$$

Por lo tanto, surge la necesidad de acotar el denominador.

Para la elección de la cota del valor exacto se ha realizado un barrido previo de un secuencia de umbrales hasta decidir en función de la pérdida de datos frente a la introducción de sesgo por valores muy altos de f .

Como se observan en la Tabla 4.3 y la Figura 4.5, cuanto más alto es el umbral, menos etiquetas quedan y más circuitos se quedan huérfanos (circuitos sin señal en ninguno de los cinco observables), pero más limpia queda la cola del objetivo.

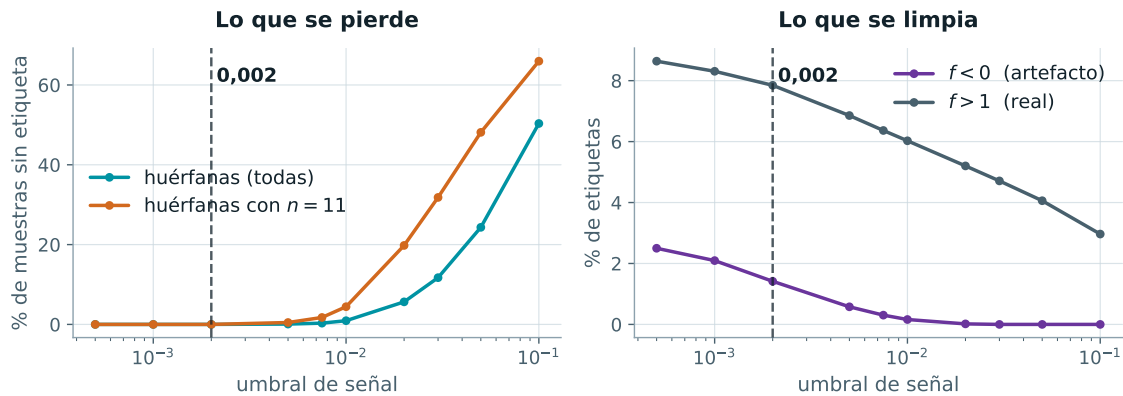
Tabla 4.3: El barrido del umbral de señal, medido sobre el conjunto de entrenamiento.

Fuente: elaboración propia.

Umbral	Etiquetas	Huérfanos	Huérf. $n=11$	$f < 0$	$f > 1$	máx. $ f $
0,0005	64.737	0,00 %	0,00 %	2,50 %	8,64 %	16,45
0,0010	63.832	0,00 %	0,00 %	2,09 %	8,31 %	11,87
0,0020	62.125	0,00 %	0,00 %	1,41 %	7,85 %	6,67
0,0050	57.647	0,08 %	0,47 %	0,58 %	6,86 %	4,31
0,0100	51.006	0,93 %	4,43 %	0,16 %	6,03 %	2,29
0,0500	23.943	24,33 %	48,13 %	0,00 %	4,06 %	1,77

Sin umbral hay 66.720 etiquetas. En negrita, el umbral elegido.

Figura 4.5: Lo que cuesta y lo que limpia cada umbral. Eje de abscisas en escala logarítmico.



Fuente: elaboración propia.

El **umbral** fijado es **0,002**, con sus consecuencias:

- **Conserva el 93,1 % de las etiquetas** (62.125 de 66.720).
- **Ningún circuito se queda huérfano** en entrenamiento ni en validación, y en el conjunto de prueba solo el 0,2 %.
- **Limpia la mayor parte de los valores máximos altos**: los factores negativos bajan al 1,41 % y la cola se recorta de un máximo de 16,45 a uno de 6,67. Un factor negativo significa que el valor medido (el ruidoso) cambió de signo respecto al exacto.
- **No penaliza a los circuitos grandes**: con este umbral el porcentaje de huérfanos es nulo en todos los tamaños.
- **Afecta de forma diferente a cada observable**

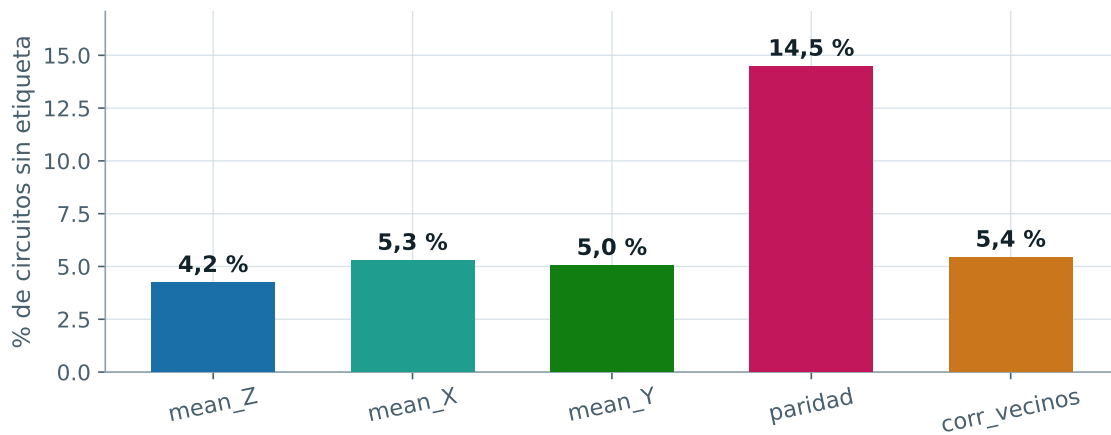
El factor negativo merece una explicación. El factor de supervivencia asume que el ruido es **multiplicativo puro**: que el valor medido es el exacto multiplicado por una fracción que sobrevive. Medido, no lo es del todo, y el modelo real es **afín**:

$$\langle O \rangle_{\text{ruidoso}} = f \cdot \langle O \rangle_{\text{exacto}} + b. \quad (4.7)$$

El término b es un desplazamiento que **no depende de cuánta señal había**, sino una constante que reciben las medidas y que son empujadas al signo opuesto.

El término b no se modela. Se asume el factor multiplicativo puro por simplificar el trabajo, y la consecuencia queda declarada: los circuitos en los que el factor cambia de signo caen fuera de lo que esta variable objetivo puede representar.

Figura 4.6: Qué porcentaje de sus casillas pierde cada observable con el umbral aplicado.



Fuente: elaboración propia. Conjunto de entrenamiento, umbral 0,002.

Aclaración importante:

*El **umbral = 0,002** se aplica tras predecir a la hora de evaluar, y no antes, reportando el número de circuitos descartados por la cota.*

4.1.4. Anatomía

Rango

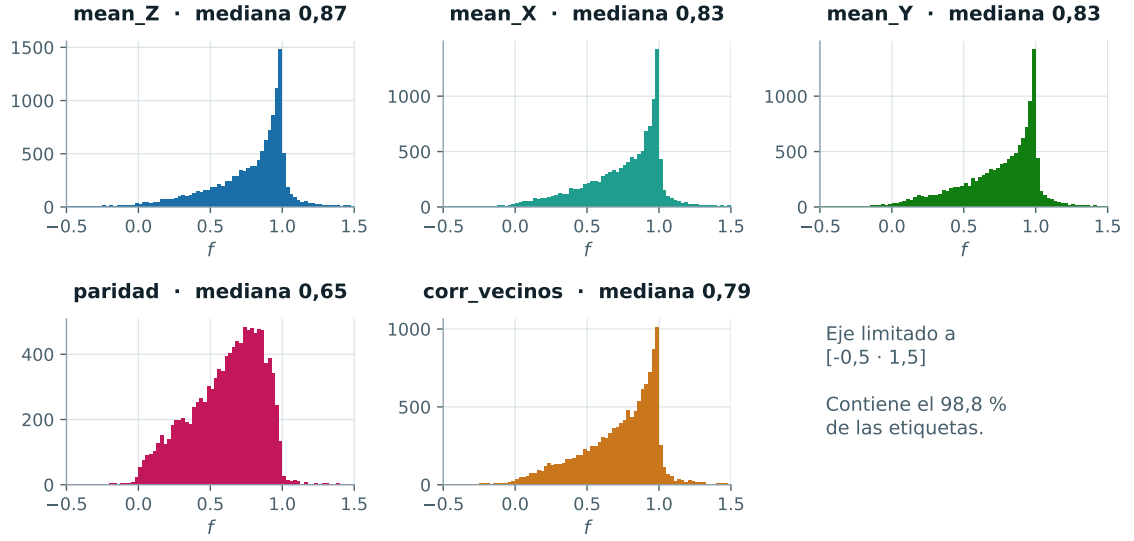
Tabla 4.4: El factor de supervivencia por observable, con el umbral de 0,002 aplicado.

Fuente: elaboración propia.

Observable	Etiquetas	Mín.	Q1	Mediana	Q3	Máx.	$f > 1$
Media de Z	12.780	-5,19	0,65	0,87	0,97	5,21	11,05 %
Media de X	12.640	-3,05	0,61	0,83	0,96	5,21	9,87 %
Media de Y	12.672	-5,88	0,60	0,83	0,96	4,64	9,86 %
Paridad	11.413	-3,16	0,44	0,65	0,81	5,03	1,31 %
Correlador vecinos	12.620	-3,54	0,56	0,79	0,94	6,67	6,47 %

Conjunto de entrenamiento.

Figura 4.7: La distribución del factor de supervivencia en los cinco observables.



Fuente: elaboración propia.

Las cinco distribuciones tienen la misma forma (Figura 4.7): la moda en uno, una cola larga hacia la izquierda que alcanza valores negativos y una cola corta por encima de uno.

La paridad es la excepción, y en las cinco columnas de la tabla. Su mediana es 0,65 frente al 0,83–0,87 del resto de observable y solo el 1,31 % de sus etiquetas supera el uno, frente al 11 % de la media de Z . Es decir, es el observable que más sufre y el que menos se amplifica.

Cotas físicas

Las dos salen del mismo sitio: el valor esperado de un observable de Pauli vive en $[-1, 1]$ (apartado 4.1.1).

Cota superior. Como $|\langle O \rangle_{\text{ruidoso}}| \leq 1$, de la definición del factor se sigue

$$|f| = \frac{|\langle O \rangle_{\text{ruidoso}}|}{|\langle O \rangle_{\text{exacto}}|} \leq \frac{1}{|\langle O \rangle_{\text{exacto}}|}. \quad (4.8)$$

Cota inferior. Sale de desarrollar la reconstrucción de la ecuación (4.4):

$$\begin{aligned} \langle O \rangle_{\text{mitigado}} &= \langle O \rangle_{\text{ruidoso}} + \Delta = \langle O \rangle_{\text{ruidoso}} + \langle O \rangle_{\text{ruidoso}} \left(\frac{1-f}{f} \right) \\ &= \langle O \rangle_{\text{ruidoso}} \left(1 + \frac{1-f}{f} \right) = \frac{\langle O \rangle_{\text{ruidoso}}}{f}. \end{aligned} \quad (4.9)$$

Es decir, **el valor mitigado es el medido dividido por el factor**. Y como ese valor mitigado es también un valor esperado de Pauli, tiene que caber en el mismo intervalo:

$$-1 \leq \langle O \rangle_{\text{mitigado}} \leq 1 \iff \left| \frac{\langle O \rangle_{\text{ruidoso}}}{f} \right| \leq 1 \iff |f| \geq |\langle O \rangle_{\text{ruidoso}}|. \quad (4.10)$$

Queda así localizada la cota inferior, y es **una por medición**: no es una constante, sino un suelo distinto para cada circuito y cada observable.

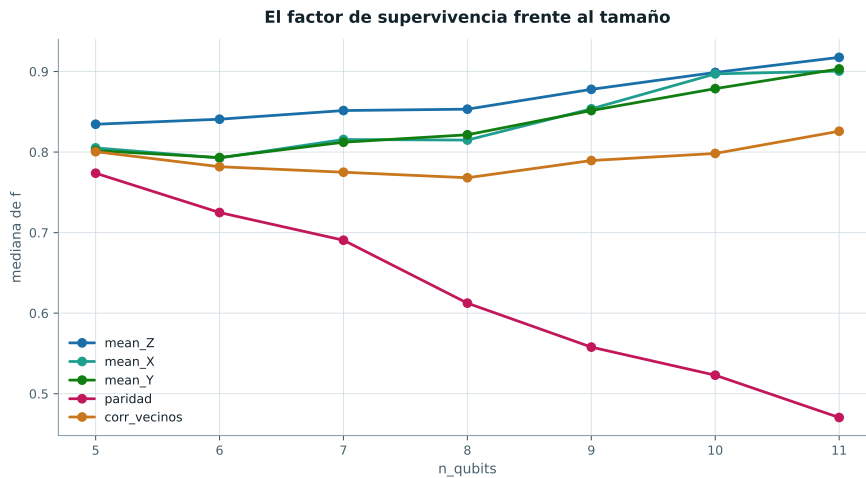
Ninguna de las 62.125 etiquetas viola ninguna de las dos, y se acerca a $|f| = |\langle O \rangle_{\text{ruidoso}}|$ solo en los circuitos con valor exacto aproximadamente uno.

La cota inferior debe aplicarse como un filtro sobre la *predicción*, pues el modelo no garantiza resultados físicos: con $\langle O \rangle_{\text{ruidoso}} = 0,01$, predecir 0,05 genera un valor razonable (0,20), pero 0,001 da 10, un absurdo físico (ya que debería de estar contenido en el intervalo $[-1, 1]$).

El factor de supervivencia frente al tamaño

El conjunto de prueba se plantea para extrapolar de once a quince qubits, así que interesa cómo se mueve la etiqueta con el tamaño dentro del rango que sí se entrena (Figura 4.8). Se concluye:

Figura 4.8: Mediana del factor de supervivencia frente al número de qubits, en el conjunto de entrenamiento.



Fuente: elaboración propia.

- Los cuatro observables promediados sobreviven algo mejor cuanto más grande es el circuito y se sufre mas del fenómeno de amplificación.

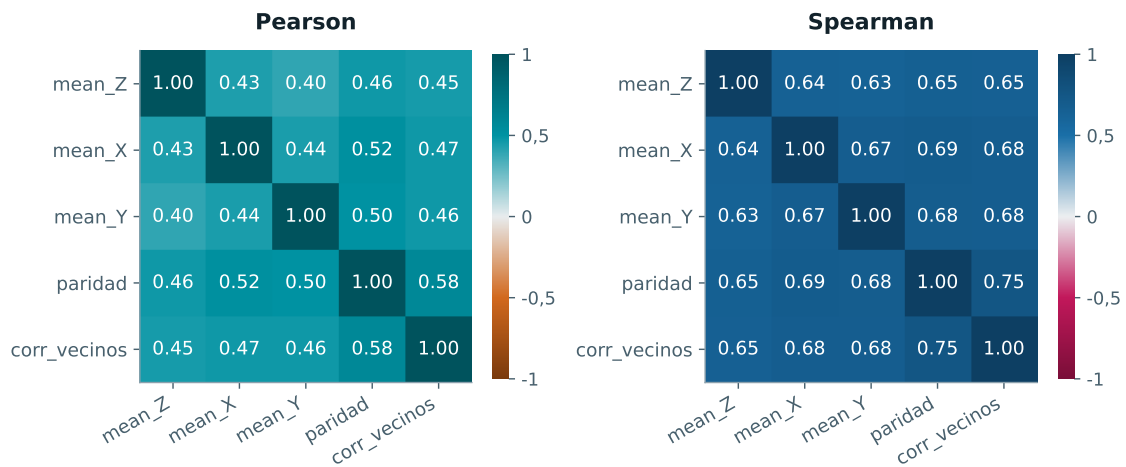
- Sin embargo, la paridad se derrumba con el aumento de qubits.

La consecuencia de esto es que los cinco objetivos son muy distintos. Entonces, al describirlos y reportar resultados es necesario tratarlos de forma independiente.

4.1.5. Correlación

Se planteó la siguiente pregunta: ¿un modelo por observable, o uno solo con cinco salidas? Un tronco compartido solo tiene sentido si los cinco tienen algún tipo de correlación.

Figura 4.9: Correlación con signo entre los cinco objetivos, medida de dos formas. Cada panel lleva su propia escala de color; las dos van de -1 a $+1$.



Fuente: elaboración propia.

La correlación de Spearman oscila entre 0,63 y 0,75, y la de Pearson entre 0,4 y 0,6. Estos datos justifican la elección de un **tronco común con cinco salidas**.

4.2. Creación del conjunto de datos

Para la obtención del conjunto de datos, la única vía posible fue fabricarlo mediante simuladores. Las razones son las siguientes:

- **Cuota.** El plan abierto de IBM concede diez minutos de procesador al mes (IBM Quantum, 2026b). Etiquetar veinte mil circuitos con ese presupuesto no era posible de forma gratuita.
- **Latencia.** Los trabajos entran en una cola compartida, y veinte mil ejecuciones con su espera no caben en el calendario establecido.
- **La etiqueta no existe en el hardware.** El modelo tiene que aprender la *diferencia* entre el resultado ideal y el ruidoso, y una máquina cuántica **solo devuelve el segundo** (Si el valor exacto se pudiera medir en la máquina, no habría nada que mitigar).

De ahí el planteamiento: un **gemelo digital**. Cada circuito se simula dos veces, una sin ruido y otra con el ruido del procesador reconstruido a partir de su calibración real de ese día. El valor ideal no se estima, se calcula; y el ruido no se inventa, se reconstruye.

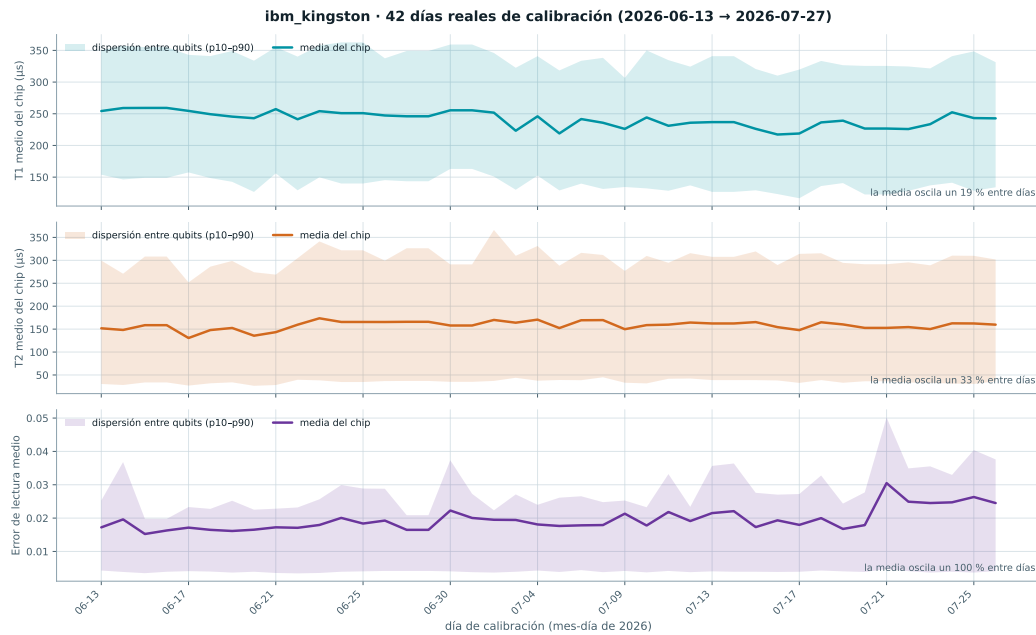
Este planteamiento fija el tamaño máximo de los circuitos. Simular el estado de n qubits exige seguir 2^n amplitudes: unos 4,3 GB para catorce qubits y 17,2 GB para quince, **por proceso**. El tope que se establece es de **quince qubits** debido a la memoria disponible.

El procesador imitado es `ibm_kingston`, de la familia **Heron r2**: 156 qubits y una puerta de dos qubits nativa que es la **CZ**, simétrica, sobre un conjunto base $\{cz, id, rz, sx, x\}$. Todos los circuitos se transpilan a ese conjunto, así que lo que el modelo ve no son las puertas que escribió el usuario, sino las que la máquina ejecutaría de verdad.

De ese procesador se descargaron 42 **días completos** de calibración, del 13 de junio al 27 de julio de 2026. Cada día aporta, por qubit, su T_1 , su T_2 , el error de lectura en las dos direcciones y la duración de la medida; y por cada puerta física del chip, su error y su duración reales.

Falta la colocación (véase el panel izquierdo de la Figura 4.11). Cada circuito se sitúa sobre una **región conexas aleatoria** de qubits físicos, distinta en cada muestra, en lugar de usar siempre los mismos. Así el modelo ve una variedad amplia de calidades de qubits. Del muestreo se excluye un solo qubit, el 146, que no tiene T_1 ni T_2 medidos ninguno de los 42 días: está averiado, y su telemetría no es un dato sino un hueco.

Figura 4.10: Recalibración diaria del procesador a lo largo de los 42 días.



Fuente: elaboración propia.

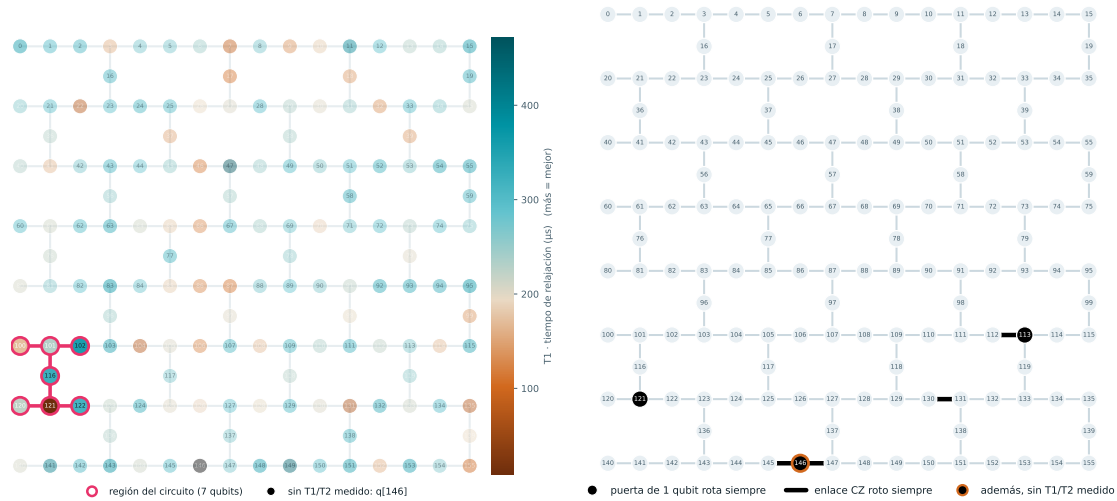
Las puertas que el chip reporta como averiadas **se mantienen** ya que es un fenómeno usual y aprendible.

El conjunto tiene 20.000 **muestras** y 18.147.608 nodos, repartidas en 16.000 de entrenamiento y validación y 4.000 de prueba. Cada muestra es un circuito de entre 5 y 15 qubits y entre 5 y 50 capas lógicas de profundidad. Los dos extremos de la profundidad están elegidos por la misma razón: por debajo de cinco capas apenas hay ruido que medir, y por encima de cincuenta la decoherencia ha destruido ya toda la señal.

El reparto entre entrenamiento y prueba se organiza para que el conjunto de prueba obligue a generalizar en **tres direcciones independientes** (Figura 4.12):

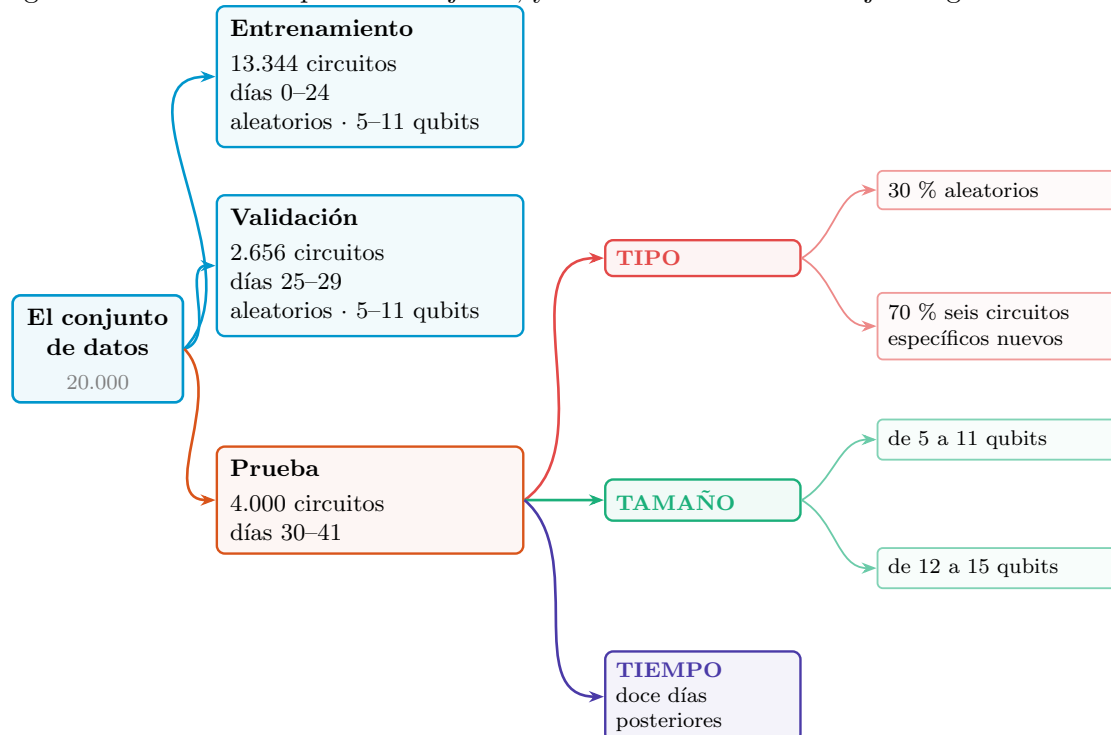
- **Tipo.** El entrenamiento es aleatorio al 100 % (del mismo modo validación) y en prueba se incluyen, a parte de circuitos aleatorios, seis circuitos con estructura diferente escogida por elección frecuente en la literatura (véase la Tabla 4.5). Si el modelo acierta en ellos, es porque aprendió física del ruido y no patrones de circuito.
- **Tamaño.** El entrenamiento llega hasta once qubits y la prueba hasta quince.
- **Tiempo.** Los días de calibración de los dos grupos son **disjuntos**: del 13 de junio al 14 de julio para entrenar, del 15 al 27 de julio para probar. El modelo se evalúa sobre un estado del chip que no vio al entrenar.

Figura 4.11: El chip por dentro. A la izquierda, la región conexas sobre la que se coloca un circuito concreto. A la derecha, las averías y la invalidez del qubit 146.



Fuente: elaboración propia.

Figura 4.12: Cómo se reparte el conjunto, y de dónde salen los tres ejes de generalización.



Fuente: Claude Code (Anthropic, 2026).

El conjunto de prueba consta de un 30 % de circuitos aleatorios que se divide por la mitad entre circuitos de hasta once qubits y circuitos de doce a quince. Los primeros permiten medir la deriva temporal con el tipo y el tamaño fijos; los segundos, la extrapolación en tamaño con el tipo fijo. El resto del conjunto de prueba son 6 circuitos específicos, elegidos por frecuencia en la literatura, y divididos de la misma forma por número de qubits. En la Tabla 4.5 aparece la justificación de la elección de estos circuitos por la literatura.

Tabla 4.5: Los siete tipos de circuito y la razón de que estén.

Fuente: elaboración propia.

Tipo	Dónde aparece	Por qué está
Aleatorio	entrenamiento y prueba	Es la física del ruido sin la estructura de ningún algoritmo. Es el 100 % del entrenamiento.
HEA	solo prueba	<i>Hardware-Efficient Ansatz</i> , el circuito de referencia de los algoritmos variacionales.
TFIM	solo prueba	Simulación de un hamiltoniano de Ising con campo transversal; referencia estándar en los bancos de prueba de mitigación.
QAOA	solo prueba	El algoritmo de optimización combinatoria más citado del estado del arte.
QFT	solo prueba	Transformada de Fourier cuántica. Aportación propia: ningún trabajo de mitigación por aprendizaje automático la evalúa de forma sistemática.
GHZ	solo prueba	El circuito de entrelazamiento por excelencia, y un caso extremo: su magnetización media es nula por construcción.
Bernstein–Vazirani	solo prueba	Salida determinista, un régimen de ruido completamente distinto al del circuito aleatorio.

4.3. Análisis exploratorio de los datos

Antes de fijar un preprocesado o una arquitectura se hizo un análisis descriptivo del conjunto de datos. Sus objetivos fueron:

- **Localizar las casillas sin etiqueta** y decidir qué hacer con ellas.
- **Análisis de relación entre las variables y el factor de supervivencia.**
- **Reducir la dimensionalidad** descartando las columnas redundantes.
- **Comprobación de escalas y asimetrías**, que son las que deciden qué transformaciones hacen falta antes de entrenar.
- **Análisis de la información que sobrevive dentro de un mismo circuito**, porque es la única que el grafo puede aprovechar y el resumen tabular no.

- **Análisis descriptivo de la forma de los grafos**

Los apartados que siguen recogen lo que se concluyó en cada uno de esos puntos.

4.3.1. Observables sin etiqueta

El umbral de señal del capítulo 4.1 deja sin etiqueta entre el 4,2 % y el 14,5 % de las casillas, según el observable, y la paridad es con diferencia la más castigada. Aun así, el 69,6 % de los circuitos de entrenamiento conserva los cinco objetivos completos y ninguno se queda sin ninguna etiqueta.

La idea de imputar las variables se descartó porque los valores que eliminaba el umbral, fijado en 0,002, eran datos que no se podían deducir de ningún otro porque a partir de ese umbral no existía ningún dato que pudiera deducir.

Decisión: no se imputa nada, y se entrena de la siguiente forma:

- Cinco modelos Ridge, uno por cada observable.
- Cinco modelos Random Forest, uno por cada observable.
- Tres modelos distintos multi-salida GNN con máscara booleana.

4.3.2. Relación de cada familia de variables con la degradación

La familia que más predice el tiempo del circuito medido frente a la decoherencia, con tres columnas.

De todas las familias, la única que no aporta apenas valor es la telemetría del hardware con trece columnas.

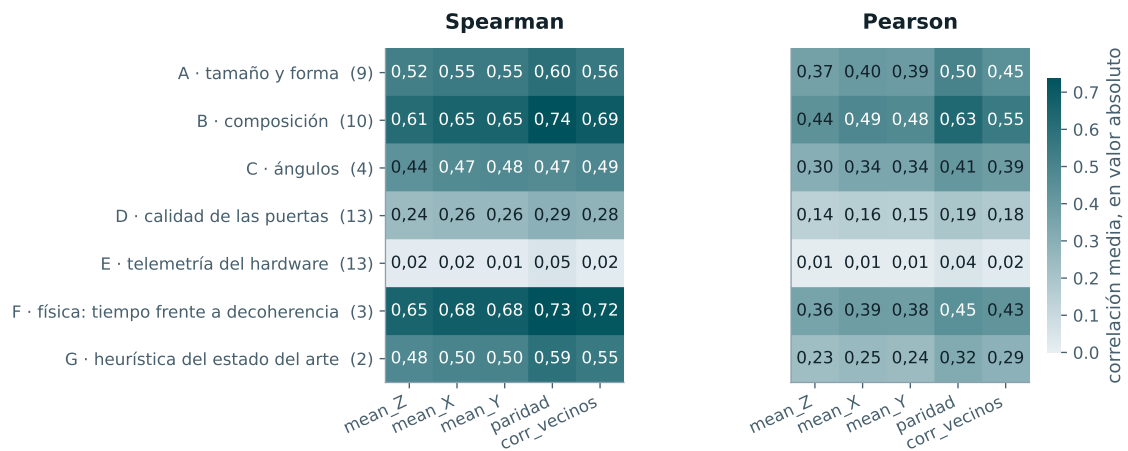
Decisión: se quedan todas, porque no afecta ni a Ridge ni a Random Forest, y porque dentro del grafo sí varían (apartado 4.3.3).

4.3.3. Telemetría de un mismo circuito y forma de los grafos

La correlación del apartado anterior se mide sobre la fila, que promedia el circuito entero. Falta por saber qué sobrevive a ese promedio, ya que es lo que separa a los dos modelos: el resumen tabular solo ve la media y el grafo ve nodo a nodo.

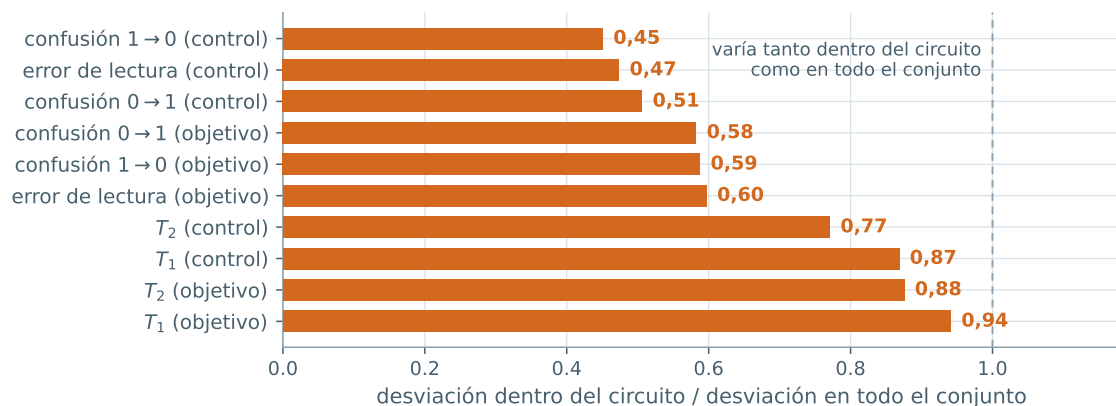
La telemetría del qubit sigue variando dentro de un mismo circuito (Figura 4.14) y es la única señal que el grafo puede aprovechar y la tabla no.

Figura 4.13: Cuánto predice cada familia de variables, por observable, medido de dos formas.



Fuente: elaboración propia.

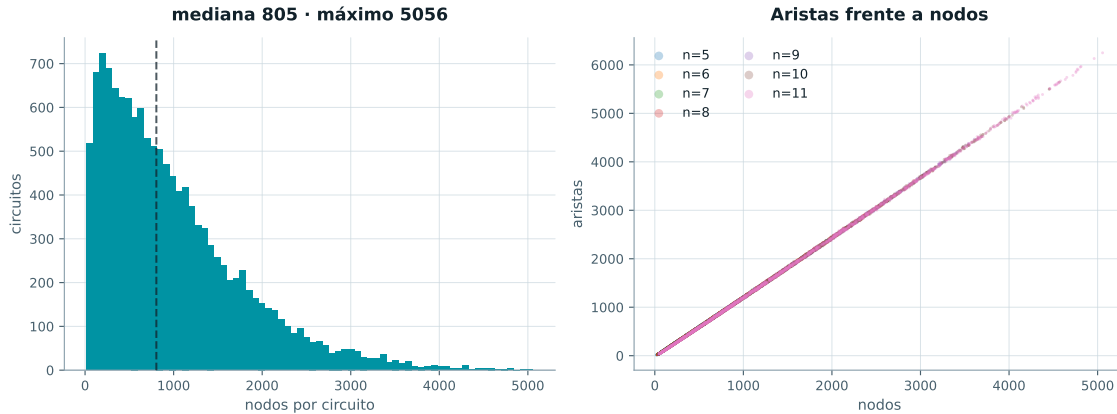
Figura 4.14: Cuánto varía la telemetría del qubit dentro de un mismo circuito.



Fuente: elaboración propia.

Los grafos de este conjunto son **muy profundos** (Figura 4.15): mediana de 805 nodos, mínimo de 18 y máximo de 5.056, con las aristas creciendo de forma lineal con los nodos. Ese factor decide una cosa del modelo de grafos: cualquier resumen que *sume* los nodos mediría el tamaño del circuito en lugar de su contenido (apartado 5.3).

Figura 4.15: Tamaño de los grafos del conjunto de entrenamiento. La línea marca la mediana.



Fuente: elaboración propia.

4.3.4. Colinealidad

De las 54 columnas candidatas (Figura 4.16) hay 11 **identidades exactas** y numerosos **bloques redundantes** con correlaciones de hasta 0,999.

Decisión: se descartan 20 columnas y quedan 34, conservando un representante de cada bloque. La matriz que queda después está en el apartado 5.1, junto al primer modelo que la consume.

4.3.5. Escalas, asimetrías y ángulos

- Las escalas son muy diferentes: 10,5 órdenes de magnitud entre la columna más pequeña y la más grande de la tabla, y 8,3 en el vector de nodo. **Decisión:** normalizar.
- Asimetría fuerte en 34 columnas, casi todas recuentos con cola larga a la derecha. **Decisión:** logaritmo antes del escalado.
- Cinco columnas tienen recorrido intercuartílico nulo, entre ellas las dos de puertas rotas, que valen cero en el 87 % de los circuitos.

Figura 4.16: Correlación entre las 54 variables candidatas, antes de descartar nada.



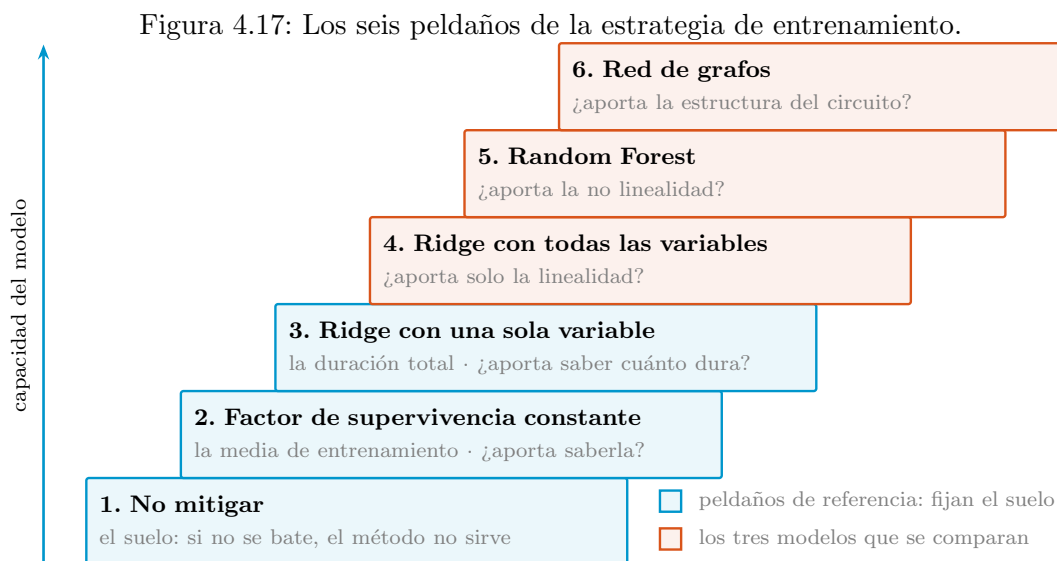
Fuente: elaboración propia.

Decisión: esas cinco no pueden llevar escalado robusto, porque dividiría por cero.

De esa regla salen dos ramas de preprocesado **independientes**: una para la matriz tabular que consumen Ridge y Random Forest, y otra para los grafos que consume la red. Ambas comparten la partición y el umbral, pero no comparten el tratamiento. Cada una se describe junto al modelo que la usa, en los apartados 5.1 y 5.3.

4.4. Estrategia de entrenamiento

La estrategia del entrenamiento se compone de los seis peldaños de la Figura 4.17. Cada uno añade una capacidad sobre el anterior, y la pregunta que lleva al lado es justamente lo que ese peldaño aísla: si al subirlo la métrica no se mueve, esa capacidad no aporta nada en este problema.



Fuente: Claude Code (Anthropic, 2026).

Los tres modelos que se comparan son los tres últimos peldaños y cada uno añade una capacidad sobre el anterior: Ridge es el **baseline** y fija el suelo, Random Forest añade no **linealidad** y está porque es el mejor modelo tabular de los que compara la referencia (Liao y cols., 2024). La red de grafos cambia la representación de los datos, y la diferencia entre ella y los dos primeros es **el valor de la estructura que la agregación descarta**.

Cabe recalcar una asimetría presente. Ridge y la red de grafos se entrenan con una **pérdida de Huber**, que penaliza los errores grandes de forma lineal y evita que los

factores de supervivencia extremos se sesgan el entrenamiento. Sin embargo, **Random Forest no la admite** y no se emplea.

4.5. Métricas y validación

El criterio general está fijado en los objetivos del capítulo 3: **batir a no mitigar**¹, y hacerlo **fuera de la distribución de entrenamiento**.

*La métrica principal es el **coeficiente de determinación sobre el factor de supervivencia**, por observable.*

$$R^2 = 1 - \frac{\sum_i (f_i - \hat{f}_i)^2}{\sum_i (f_i - \bar{f})^2}. \quad (4.11)$$

Además, para evaluar la desviación absoluta sobre la propia cantidad aprendida (f), se emplea el Error Absoluto Medio (MAE):

$$\text{MAE}_f = \frac{1}{n} \sum_i |f_i - \hat{f}_i|. \quad (4.12)$$

Finalmente, para medir la fidelidad del valor cuántico reconstruido y evaluar la utilidad práctica frente a la línea base del hardware, se utiliza la **mejora relativa**:

$$\text{Mejora Relativa} = 1 - \frac{\text{MAE}(\langle O \rangle_{\text{mitigado}})}{\text{MAE}(\langle O \rangle_{\text{ruidoso}})}. \quad (4.13)$$

Esta métrica es fundamental para evaluar hasta qué punto la mitigación ha sido efectiva. Un valor de 1 corresponde al escenario ideal, en el que el error se corrige por completo. Los valores comprendidos entre 0 y 1 indican la proporción de error del hardware que se ha conseguido eliminar. Cuando la métrica toma el valor 0, significa que el modelo no aporta ninguna mejora respecto a no aplicar mitigación. Por el contrario, los valores negativos revelan que la corrección resulta contraproducente, ya que introduce más error del que logra reducir.

La evaluación aplica además la **cota inferior del factor** deducida en el apartado 4.1.4: una predicción por debajo de $|\langle O \rangle_{\text{ruidoso}}|$ produciría un valor corregido imposible, así que antes de reconstruir se sube hasta la cota y se reporta qué porcentaje de predicciones la

¹Es decir, lograr que el error de la estimación cuántica corregida ($\langle O \rangle_{\text{mitigado}}$) sea estrictamente menor que el error de la ejecución ruidosa original ($\langle O \rangle_{\text{ruidoso}}$) obtenida directamente del procesador.

tocó.

$$\hat{f}_{\text{final}} = \text{máx} \left(\hat{f}, |\langle O \rangle_{\text{ruidoso}}| \right), \quad (4.14)$$

donde $\langle O \rangle_{\text{ruidoso}}$ es el valor esperado medido con ruido.

Por último, todo lo anterior se reporta **desglosado por los tres ejes de generalización** (tipo, tamaño y tiempo) descritos en el apartado 4.2.

5. Desarrollo de la comparativa

Los tres modelos resuelven el mismo problema con la misma partición, el mismo umbral de señal y las mismas métricas, fijados todos en el capítulo 4. El cambio entre un modelo y otro es la preparación de los datos de entrada, y es eso lo que recoge este capítulo: de cada modelo, los datos que consume, el preprocesado correspondiente y los hiperparámetros seleccionados.

Una aclaración sobre las cifras que aparecen. Todas las cifras de este capítulo son del conjunto de **validación**, ya que son el criterio con el que se eligió cada hiperparámetro. Ninguna es de prueba: el conjunto de prueba no se toca hasta el capítulo 6.

5.1. Ridge

Una regresión lineal con penalización L_2 (Hoerl y Kennard, 1970). Es el **baseline** de la comparativa: fija el suelo que los otros dos tienen que batir para justificar su coste.

El modelo recibe una fila por circuito. Se ajusta **un modelo por observable**, cinco en total, y cada uno descarta las filas que no tienen etiqueta de *su* observable, porque el umbral de señal del apartado 4.1.3 deja sin objetivo a muestras distintas según el observable. La Tabla 5.1 recoge con cuántas filas se entrena cada uno.

Tabla 5.1: Filas con etiqueta por observable, en entrenamiento y en validación.

		media Z	media X	media Y	paridad	corr. vec.
Fuente: elaboración propia.	Entrenamiento	12.780	12.640	12.672	11.413	12.620
	Validación	2.566	2.526	2.551	2.292	2.516

Sobre 13.344 y 2.656 circuitos respectivamente. La paridad es la que más pierde por el umbral establecido

El preprocesado

El conjunto tabular trae 54 columnas y el modelo ve 34. Tres decisiones, en este orden:

- **Se descartan 20 columnas redundantes**, conservando un representante de cada bloque correlado (apartado 4.3.4). La Figura 5.1 es la matriz que queda después.
- **Transformación logarítmica** a toda columna con asimetría mayor que uno en módulo.
- **Escalado** de todas las columnas.

Ninguna de las tres se ajusta con validación. Las constantes salen solo de entrenamiento.

Figura 5.1: Correlación entre las 34 variables que quedan tras el descarte.



Fuente: elaboración propia.

No se usan mínimos cuadrados ordinarios precisamente por esa matriz: aun después del descarte queda correlación entre columnas, y la penalización L_2 reparte el peso entre las correlacionadas.

Los hiperparámetros

El único es la constante de regularización, barrida en diez valores de 10^{-3} a 10^6 y elegida por el coeficiente de determinación sobre validación. La Tabla 5.2 recoge el elegido para cada observable.

Los valores son altos, entre 100 y 1.000 sobre columnas ya escaladas, lo que dice que el ajuste necesita bastante penalización. La columna de la izquierda es el peldaño de una sola variable de la Figura 4.17: el mismo modelo viendo solo la duración total, para que la

Tabla 5.2: Constante de regularización elegida para cada observable.

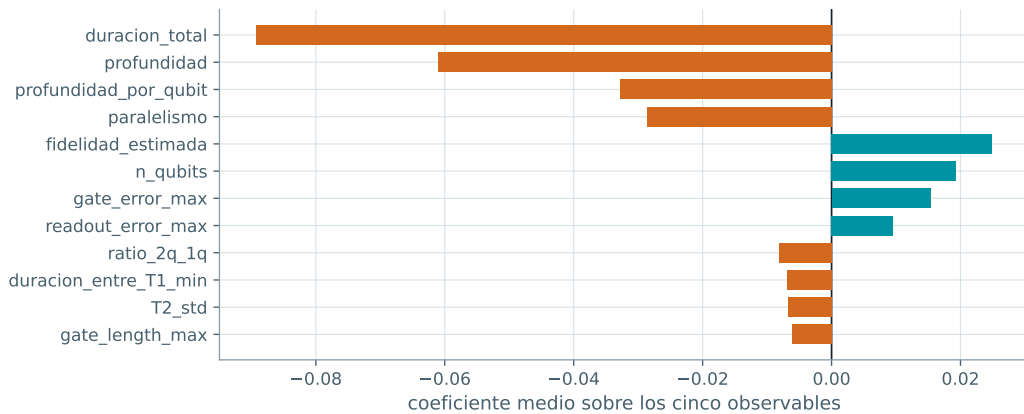
	Observable	$\alpha \cdot \text{una variable}$	$\alpha \cdot 34 \text{ variables}$
Fuente: elaboración propia.	media Z	100	100
	media X	1.000	1.000
	media Y	1.000	1.000
	paridad	100	1.000
	corr. de vecinos	1.000	100

Elegida por R^2 sobre validación, en una rejilla de diez valores.

diferencia entre los dos peldaños sea únicamente cuántas columnas entran.

La Figura 5.2 recoge los doce coeficientes de mayor magnitud. Se muestran **con su signo**, que es lo único que este modelo da y los otros dos no: la dirección en la que cada variable empuja al factor de supervivencia.

Figura 5.2: Los doce coeficientes mayores de Ridge, con su signo, promediando los cinco observables.



Fuente: elaboración propia.

5.2. Random Forest

Un bosque de árboles de decisión (Breiman, 2001). Añade sobre Ridge una sola cosa, **la no linealidad**, y está en la comparativa porque es el mejor modelo tabular de los que compara la referencia del área (Liao y cols., 2024).

Conviene decir de entrada su límite: **un árbol no extrapola**. Su predicción para una entrada fuera del rango visto es la de la hoja más cercana, así que en el eje de tamaño parte con una desventaja estructural que no es un defecto del ajuste.

El modelo recibe exactamente los mismos datos que Ridge: las mismas 34 columnas, un modelo por observable y los mismos descartes de la Tabla 5.1.

El preprocesado

También el mismo, y aquí hay una decisión que conviene justificar. A un árbol **no le afecta** el escalado ni la transformación logarítmica. Aun así **se le aplican las dos**. El motivo es mantener una misma entrada a ambos modelos.

Los hiperparámetros

La rejilla fija 300 árboles y barre tres ejes: la profundidad máxima ($\{\text{sin límite}, 6, 12, 24\}$), las hojas mínimas ($\{1, 5, 20, 50\}$) y las variables por división ($\{\sqrt{\cdot}, 0,5, 1,0\}$). Son 48 combinaciones por observable, 240 ajustes en total. La Tabla 5.3 recoge la ganadora de cada uno.

Tabla 5.3: Hiperparámetros elegidos para cada observable.

Fuente: elaboración propia.

Observable	Profundidad	Hojas mínimas	Variables por división
media Z	6	50	0,5
media X	24	50	0,5
media Y	12	50	0,5
paridad	6	5	0,5
corr. de vecinos	6	5	1,0

Los cinco con 300 árboles. Elegidos por R^2 sobre validación entre 48 combinaciones.

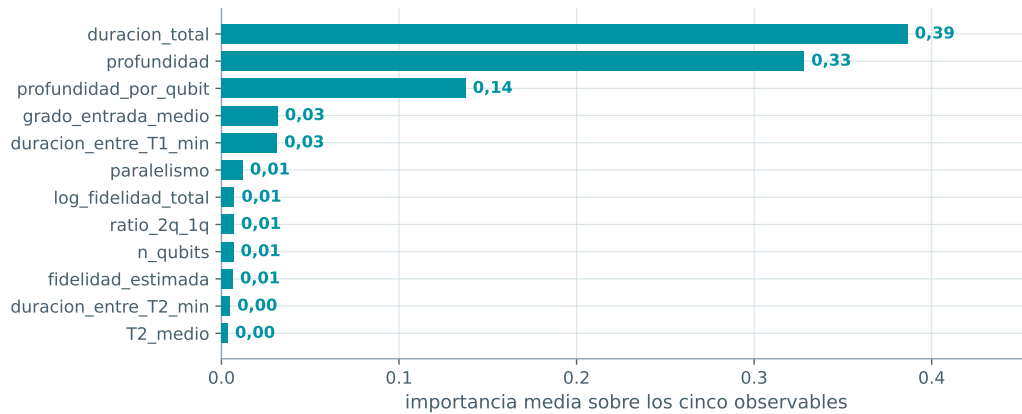
Los óptimos salen **poco profundos**: profundidad 6 en tres de los cinco observables, y hojas de 50 muestras en los tres primeros. Un bosque al que se permite crecer sin restricciones termina sobreajustándose, lo que sugiere que la información contenida en estas variables es relativamente simple. La única excepción es la media de X , cuyo rendimiento óptimo se alcanza con una profundidad de 24.

La Figura 5.3 y la Figura 5.2 transmiten la misma conclusión: la **duración total** y la **profundidad** concentran la mayor parte de la capacidad predictiva del modelo, con importancias de 0,39 y 0,33, respectivamente, frente al 0,03 de la siguiente variable. A pesar de basarse en criterios de ajuste diferentes, ambos modelos se basan sus predicciones principalmente en estas dos características.

5.3. La red de grafos

Para el último eslabón de la escalera, se ha seleccionado una red de paso de mensajes sobre el grafo del circuito. Requiere de una justificación detallada porque es la que se aparta más de la arquitectura de referencia del área.

Figura 5.3: Las doce variables más importantes para Random Forest, promediando los cinco observables.



Fuente: elaboración propia.

El trabajo más cercano (Bao y cols., 2025) usa un *graph transformer sin* paso de mensajes. Aquí se descartó por cómputo: la atención es cuadrática en el número de nodos y los grafos de este conjunto llegan a 5.056, de modo que la máquina disponible no daba para ello.

Lo que el modelo recibe no es una fila, sino **el circuito entero**: un grafo dirigido con una puerta por nodo. Los grafos van de 18 a 5.056 nodos y **entran completos**, sin truncar. Cada nodo llega con 21 dimensiones.

A diferencia de los dos tabulares, aquí hay **un solo modelo con cinco salidas**, no cinco modelos. El tronco es compartido y la última capa tiene cinco neuronas, una por observable. Eso obliga a dos cosas que en los tabulares no hacían falta y que se detallan más abajo: una pérdida enmascarada y la estandarización de los cinco objetivos.

El preprocesado del grafo

Es **independiente** del tabular. Comparten la partición y el umbral de señal, pero no comparten ni una transformación. Cuatro decisiones:

- **El vector de nodo pasa de 25 dimensiones a 21.** Se podan cinco y se añade una por la codificación del ángulo. Los motivos de la poda, en la Tabla 5.4.
- **El ángulo se sustituye por su seno y su coseno.** Los ángulos no vienen envueltos (van de $-12,43$ a $18,87$ radianes)
- **Los ceros de relleno se reimponen.** En el 78,88 % de los nodos las seis dimensiones del qubit objetivo valen cero porque no hay segundo qubit. Al estandarizar

dejarían de valer cero e introducirían información falsa, así que la escala se ajusta con los nodos de dos qubits y después se vuelve a poner el cero exacto.

- **Las constantes de escalado se congelan en un fichero y se leen.**

Tabla 5.4: Las cinco dimensiones podadas del vector de nodo, y su motivo.

Fuente: elaboración propia.

Dimensión	Motivo
tipo id	Cero nodos de este tipo
duración de medida (control)	Constante en $2,18 \mu s$
aridad	Idéntica a otra dimensión
duración de medida (objetivo)	Redundante
día	Fuera del rango de entrenamiento el 100 % de las veces

La arquitectura

La Figura 5.4 la resume. Cuatro piezas, y la tercera es la que sostiene el modelo:

1. **Aristas estrictamente dirigidas.** El ruido se acumula hacia adelante en el tiempo.
2. **Conexiones residuales¹ y normalización de capa** tras cada actualización de mensajes, para mitigar el *over-smoothing*² provocado por el resumen global.
3. **Un resumen global bidireccional:** lee de todos los nodos y *devuelve* a todos. Está implementado como un vector por grafo con sus **propios pesos**, separado de la convolución.
4. **Lectura final:** el vector de resumen concatenado con el *pooling* global de los nodos.

La idea del resumen global se toma del mismo trabajo (Bao y cols., 2025), con un cambio. Allí es **unidireccional**, y en un *transformer* basta, porque su atención ya conecta todo con todo. Sin embargo, en una red de paso de mensajes se tiene que recorrer los nodos. El resumen es la única vía al largo alcance.

El resumen se calcula con **media**

¹Técnica arquitectónica que suma el estado previo de un nodo a su nuevo valor tras procesar la información de la capa ($h_{l+1} = \text{Capa}(h_l) + h_l$). Actúa como un ancla de memoria que garantiza que las características locales de la puerta cuántica original no se diluyan bajo el resumen global.

²Fenómeno en las redes neuronales de grafos (GNNs) en el que, tras sucesivas operaciones de agregación de información procedente del vecindario o del contexto global, las representaciones de los nodos tienden a converger hacia valores muy similares. Como consecuencia, se pierde la capacidad de distinguir características locales y estructurales de cada nodo, reduciendo su poder discriminativo y dificultando la diferenciación entre ellos.

Figura 5.4: La arquitectura. El resumen global lee de todos los nodos y les devuelve: son las dos flechas, no una.



Fuente: Claude Code (Anthropic, 2026).

La pérdida

Se añaden estas dos decisiones por necesidad de los datos:

- **Huber en lugar del error cuadrático.** El factor de supervivencia llega a valores de seis y siete con mediana menor que uno, y con error cuadrático esos pocos extremos sesgan el ajuste. Hay precedente publicado para la misma decisión en este problema (Aktar y cols., 2024). El umbral donde Huber pasa de cuadrático a lineal se fijó en 0,5.
- **Enmascarada.** El objetivo no tiene etiqueta donde no hay señal, y dónde falta cambia según la muestra y según el observable. Como el modelo tiene cinco salidas compartiendo tronco, la pérdida se calcula solo sobre las casillas con etiqueta y se normaliza por cuántas había, no por el total.

Los cinco objetivos se **estandarizan** antes de entrenar, porque el tronco es compartido y el de mayor varianza se llevaría el ajuste. Las constantes salen solo de entrenamiento y viajan dentro del punto de control: sin ellas las predicciones no se pueden devolver a la escala de f .

El barrido

Se barre sobre la convolución más ligera y después se trasladan los ganadores a las otras dos. La rejilla recorre la tasa de aprendizaje ($\{3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}\}$), la dimensión oculta ($\{128, 192, 256\}$) y el número de capas ($\{4, 6\}$): dieciocho configuraciones, con lotes

de 32 grafos, regularización de 10^{-5} y parada temprana con paciencia de quince épocas. La Tabla 5.5 recoge las cinco mejores y la peor.

Tabla 5.5: Las mejores configuraciones del barrido, y la peor, sobre SAGEConv.

	Tasa	Oculto	Capas	Épocas	Mejor época	Minutos
	10^{-3}	256	4	29	14	149,4
	$3 \cdot 10^{-4}$	256	6	29	14	215,1
Fuente: elaboración propia.	$3 \cdot 10^{-3}$	128	6	22	7	56,2
	$3 \cdot 10^{-3}$	192	6	20	5	84,8
	10^{-3}	256	6	43	28	298,5
	10^{-3}	192	4	22	7	61,6

Ordenadas por R^2 medio sobre validación; la última fila es la peor de las dieciocho. Entre la mejor y la peor hay 23 milésimas de R^2 y un factor de cinco en tiempo de cómputo.

La lectura del barrido es que **la rejilla apenas separa**: entre la mejor configuración y la peor hay veintitrés milésimas de coeficiente de determinación, mientras que el tiempo de cómputo se multiplica por cinco. La elegida es la primera fila: tasa 10^{-3} , dimensión oculta 256 y cuatro capas.

Las tres convoluciones

Se prueban **escalando**, en orden de complejidad creciente (Tabla 5.6): se sube de peldaño solo si el anterior se queda corto, de modo que si la atención local mejora se sabe que aporta, y si no, también.

Tabla 5.6: Las tres convoluciones que se prueban y la configuración con la que se entrenó cada una.

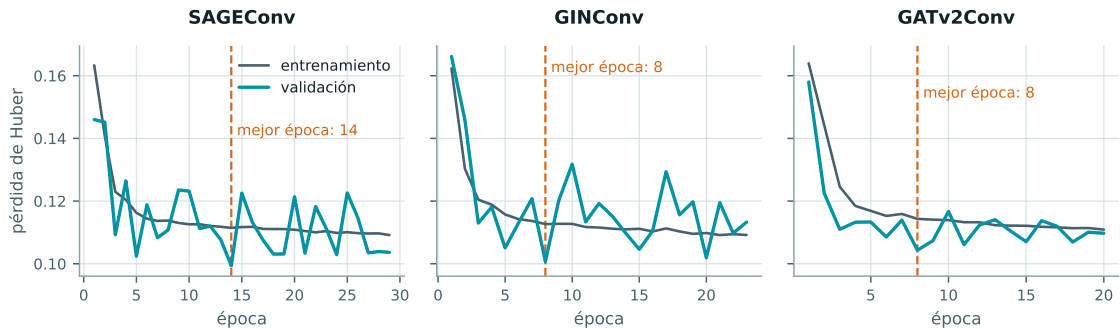
Fuente: elaboración propia.					
Convolución	Qué añade	Oculto	Tasa	Parámetros	
SAGEConv (Hamilton, Ying, y Leskovec, 2017)	Agrega los vecinos por media. Ligera y estable. Es la primera	256	10^{-3}	1.060.613	
GINConv (Xu, Hu, Leskovec, y Jegelka, 2019)	Suma estricta más un perceptrón interno. La más expresiva; distingue mejor topologías parecidas	256	$3 \cdot 10^{-4}$	1.061.641	
GATv2Conv (Brody, Alon, y Yahav, 2022)	Atención local : aprende a pesar más a los vecinos con peor telemetría	160	10^{-3}	1.039.205	

Las tres con cuatro capas.

Las tres se entrenan con el **mismo presupuesto de parámetros**, alrededor de 1,05 millones, con un 2 % de diferencia entre la mayor y la menor. Cabe recalcar que es importante establecer presupuestos parecidos para que la diferencia marcada sea la arquitectónica. Es la razón de que GATv2Conv lleve dimensión oculta 160 y no 256: su mecanismo

de atención tiene más pesos por unidad de anchura, y a 256 se habría ido muy por encima del presupuesto.

Figura 5.5: Pérdida de entrenamiento y de validación, época a época, para las tres convoluciones.



Fuente: elaboración propia. La línea vertical marca la época del punto de control que se conservó.

La Figura 5.5 muestra las curvas de entrenamiento y validación de los tres modelos. Se pueden destacar 2 aspectos. Primero, la parada temprana funciona correctamente: el mejor modelo se obtiene en las épocas 14, 8 y 8, y a partir de esos puntos la pérdida de validación deja de mejorar. Segundo, no se observa sobreajuste en ninguno de los tres casos, ya que las curvas de validación no muestran un aumento sostenido respecto al entrenamiento.

El entrenamiento se hizo en una máquina con tarjeta gráfica en la nube (Google, 2026b), porque en local no hay ninguna.

6. Discusión y análisis de resultados

Este capítulo mide lo que los tres modelos del capítulo 5 hacen sobre el conjunto de prueba. Se separa en dos partes: primero lo que el modelo **aprende** (el factor de supervivencia, predicho antes de ejecutar) y después lo que el modelo **sirve**, es decir, cuánto se acerca al valor exacto el observable una vez corregido.

Además, se reportan el porcentaje de circuitos que se ha salvado de la **cota inferior del factor de supervivencia**.

6.1. Resultados sobre el factor de supervivencia

Antes de mirar la prueba conviene revisar la medida en validación Tabla 6.1: mismos tipos de circuito y mismo rango de tamaños que el entrenamiento.

Tabla 6.1: Coeficiente de determinación sobre el factor, en validación.

Fuente: elaboración propia.

	media Z	media X	media Y	paridad	corr. vec.
No mitigar	-0,43	-0,61	-0,53	-1,78	-0,84
Factor constante	-0,00	-0,00	-0,00	-0,01	-0,01
Ridge · 1 variable	0,39	0,49	0,44	0,62	0,59
Ridge · 34 variables	0,41	0,52	0,45	0,65	0,60
Random Forest	0,42	0,53	0,46	0,67	0,61
GNN · SAGE	0,41	0,52	0,46	0,64	0,60
GNN · GIN	0,41	0,51	0,45	0,63	0,59
GNN · GATv2	0,40	0,52	0,45	0,60	0,57

Entre 2.282 y 2.519 circuitos por celda, según el observable.

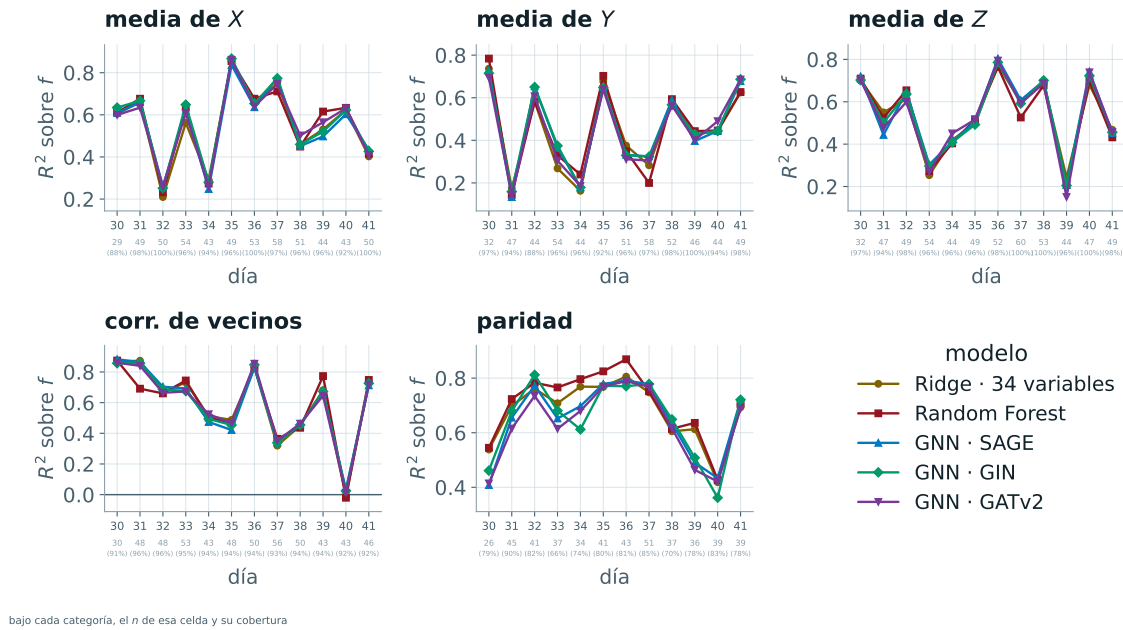
La escalera dice tres cosas. La primera, que **no mitigar es peor que no hacer nada**, de modo que el listón de verdad no es el cero sino un factor constante. La segunda, que **casi todo lo aprendible se aprende con una sola variable**: la duración total del circuito ya consigue entre el 39 % y el 62 % de la métrica, y las treinta y tres restantes añaden tres puntos como mucho (y en el correlador de vecinos, tres milésimas). Y la tercera, que **dentro de la distribución de entrenamiento los cinco modelos empatan** y aunque Random Forest gana por un margen que no se ve en una gráfica.

Para desempatar entre los 5 modelos se comprobará en el conjunto de prueba.

6.1.1. Eje temporal: la deriva del hardware

La deriva se mide **en limpio**: solo circuitos aleatorios y de hasta once qubits, de modo que lo único que varía entre celdas es el día de calibración (Figura 6.1). Sin fijar el tipo y el tamaño, lo que se estaría midiendo es otro eje.

Figura 6.1: Coeficiente de determinación sobre el factor, por día de calibración, sobre circuitos aleatorios de hasta once qubits.



Fuente: elaboración propia. Entre 29 y 60 circuitos por celda.

El resultado es que **no hay deriva medible en doce días**: los cinco modelos suben y bajan juntos, con diferencias entre ellos.

El modelo que destaca un poco por encima de un observable en concreto (paridad) es Random Forest, pero no es concluyente.

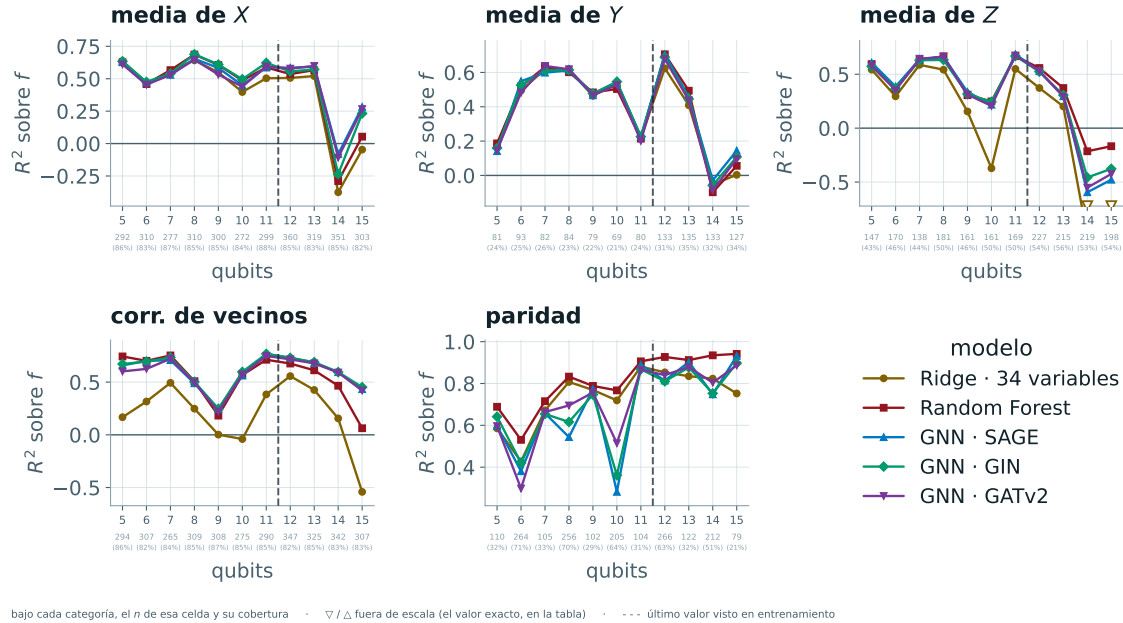
6.1.2. Eje de tamaño: extrapolación a circuitos mayores

El entrenamiento llega hasta once qubits y la prueba hasta quince, así que los cuatro últimos puntos de la Figura 6.2 son extrapolación. Cabe destacar que Random Forest no puede extrapolar por construcción.

Dentro del rango conocido (5-11 qubits) los cinco modelos vuelven a moverse juntos, con Ridge algo por debajo y separándose del resto en el observable correlador de vecinos. Lo interesante empieza a la derecha de la línea:

- La media de X es donde la red de grafos gana. A catorce y quince qubits

Figura 6.2: Coeficiente de determinación sobre el factor, por número de qubits. La línea a trazos marca el último tamaño presente en el entrenamiento.



Fuente: elaboración propia. Entre 69 y 360 circuitos por celda.

saca a Random Forest 0,21 y 0,23 puntos de coeficiente. Es el único sitio de todo el trabajo donde la estructura del circuito sirve para predecir más que el resumen.

- **La media de Z se hunde para todos.**
- **La paridad aguanta entera.** Se mantiene entre 0,89 y 0,95 hasta los quince qubits, y es el único observable que cruza la frontera sin sufrir, encabezado por Random Forest.

En resumen, **existe ventaja del grafo pero local**: aparece solo al extrapolar en tamaño, y solo en el observable donde queda señal que extrapolar.

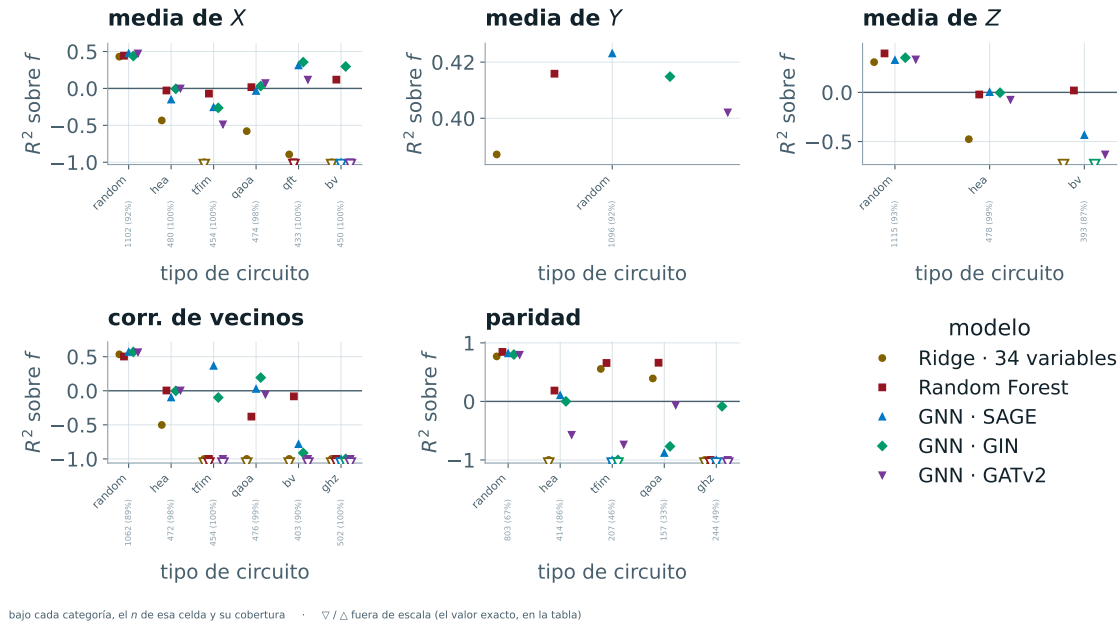
6.1.3. Eje de tipo: algoritmos nunca vistos

Es el eje más exigente de los tres. El entrenamiento es aleatorio al cien por cien, así que los seis tipos con estructura del conjunto de prueba se evalúan a ciegas, y dentro de cada tipo de observable.

Por lo general, **el coeficiente de determinación cae en la mayoría de los casos**. Y se destacan dos hallazgos

- **En circuitos aleatorios (el caso conocido) todos funcionan**, entre 0,31 y 0,57,

Figura 6.3: Coeficiente de determinación sobre el factor, por tipo de circuito. Las celdas que se salen del eje van marcadas.



Fuente: elaboración propia. Entre 157 y 1.115 circuitos por celda.

con Random Forest y las redes de grafos encabezan el primer puesto por centésimas.

- **La red de grafos gana en pocos casos** por encima de Random Forest: predice mejor qft y bv en media X, en media de Y con random y tfim en correlación de vecinos.
- **En el resto de tipos con estructura nadie aprende gran cosa.** Random Forest gana por lo general en el resto.

Sumando los tres ejes, el veredicto sobre la métrica que decide es **Random Forest**: gana en validación, va en cabeza dentro del rango de tamaños conocido, y en el eje temporal gana con soltura. Las redes de grafos lo superan en dos sitios concretos: grandes la media de X al extrapolar en tamaño, y algunos circuitos específicos y de un observable concreto que no tienen relevancia. Entonces, se responde a la pregunta con la que se cerró el capítulo 4.1:

*La estructura del circuito **sí** aporta información que el resumen agregado descarta, pero solo en la extrapolación del número de qubits. En el régimen conocido, treinta y cuatro columnas bastan.*

6.2. Resultados sobre la mitigación

Este apartado mide la distancia entre lo bien que predice el factor y lo bien que corrige. Por construcción, un valor negativo significa que **habría sido mejor no tocar nada**.

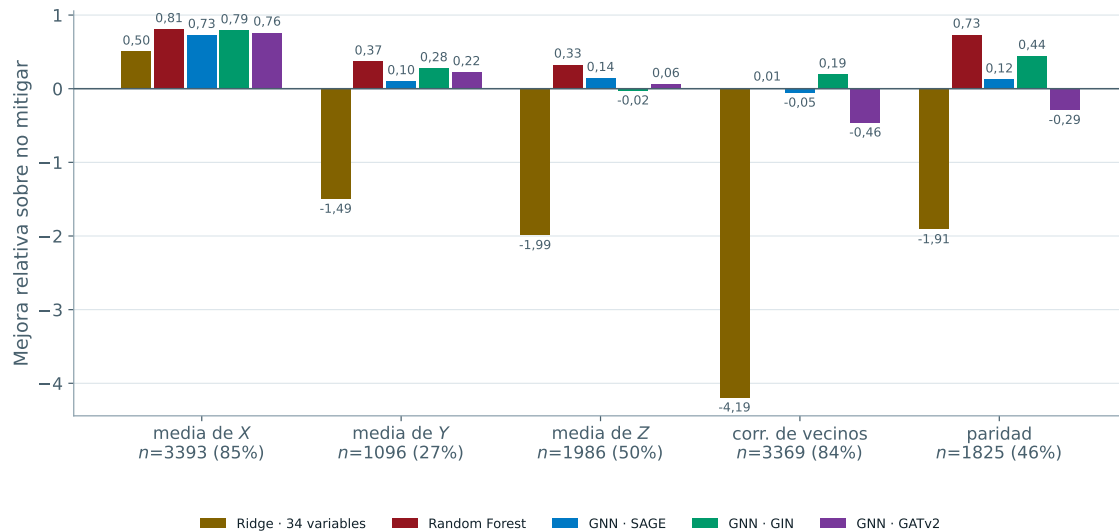
Tabla 6.2: Mejora relativa del observable corregido frente a no corregir.

Fuente: elaboración propia.

Observable	n	Sin mitigar	Mejora relativa				
		MAE	Ridge	RF	SAGE	GIN	GATv2
media Z	1.986	0,0046	-1,99	+0,33	+0,14	-0,02	+0,06
media X	3.393	0,0177	+0,50	+0,81	+0,73	+0,79	+0,76
media Y	1.096	0,0044	-1,49	+0,37	+0,10	+0,28	+0,22
paridad	1.825	0,0111	-1,91	+0,73	+0,12	+0,44	-0,29
corr. vec.	3.369	0,0072	-4,19	+0,01	-0,05	+0,19	-0,46

Conjunto de prueba.

Figura 6.4: Mejora relativa sobre no mitigar, por observable y modelo. El cero es exactamente no corregir.



Fuente: elaboración propia. Conjunto de prueba.

El resultado que más dice del problema es el de Ridge. Su coeficiente sobre el factor era **positivo en los cinco observables** (entre 0,41 y 0,65 en validación) y aun así, aplicado, **empeora el resultado en cuatro de los cinco**. Por tanto, **un buen ajuste sobre el factor no garantiza una buena mitigación**, y este trabajo lo mide en lugar de suponerlo.

En cuanto al resto. **Random Forest es el único que nunca empeora**, de +0,01 a +0,81, y es el mejor en cuatro de los cinco observables. Entre las redes, GIN es la más consistente y la única que aporta algo en el correlador de vecinos. Y la media de X es el

único observable donde los cinco modelos ayudan, con mejoras de entre el 50 % y el 81 %.

Queda un matiz que una media no puede dar. Medida como **proporción de circuitos que mejoran**, Ridge corrige bien el 73 % de las veces en la paridad y aun así su mejora media es de $-1,91$. GIN mejora más circuitos (93 % frente a 89 %) y sin embargo su mejora media es menor ($+0,44$ frente a $+0,73$) de Random Forest, porque acierta más veces pero falla más fuerte. Las dos métricas cuentan historias distintas.

Uniando las dos mitades del capítulo, el trabajo termina con dos conclusiones que no coinciden. Sobre la métrica que decide (la capacidad de predecir antes de ejecutar) el resumen tabular basta en el régimen conocido y la estructura del grafo solo gana al extrapolar. Sobre la aplicación, el orden es el mismo pero la distancia entre los modelos es mayor.

Tabla 6.3: Coeficiente de determinación sobre el factor, por eje de generalización.

Fuente: elaboración propia.

	media Z	media X	media Y	paridad	corr. vec.
<i>Eje de tipo</i> — circuitos estructurados, hasta 11 qubits					
Ridge · 34 variables	$-1,32$	$-0,15$	—	0,43	$-4,22$
Random Forest	$-0,01$	0,18	—	0,70	$-0,09$
GNN · SAGE	$-0,02$	0,09	—	$-0,45$	$-0,09$
GNN · GIN	$-0,15$	0,24	—	$-0,23$	0,05
GNN · GATv2	$-0,10$	$-0,07$	—	0,06	$-0,28$
<i>Eje de tamaño</i> — circuitos aleatorios, de 12 a 15 qubits					
Ridge · 34 variables	0,14	0,35	0,38	0,60	0,56
Random Forest	0,30	0,37	0,43	0,82	0,51
GNN · SAGE	0,17	0,43	0,45	0,80	0,62
GNN · GIN	0,22	0,36	0,43	0,74	0,62
GNN · GATv2	0,19	0,42	0,42	0,74	0,61
<i>Eje temporal</i> — circuitos aleatorios, hasta 11 qubits					
Ridge · 34 variables	0,54	0,54	0,39	0,68	0,49
Random Forest	0,53	0,55	0,39	0,71	0,48
GNN · SAGE	0,55	0,54	0,39	0,65	0,49
GNN · GIN	0,54	0,55	0,39	0,66	0,49
GNN · GATv2	0,54	0,54	0,38	0,64	0,49

Cada eje fija los otros dos, de modo que los tres subconjuntos son disjuntos y una caída se puede atribuir a un eje y no a la mezcla. La raya en la media de Y del eje de tipo no es un hueco: ningún circuito estructurado define ese observable. En negrita, el mejor de cada columna antes de redondear.

La Tabla 6.4 cierra el capítulo con las dos métricas juntas, sobre el conjunto de prueba entero y sin separar por ejes.

Tabla 6.4: Resumen de la comparativa: los ocho peldaños frente a las dos métricas, por observable, sobre el conjunto de prueba completo (sin distinción de eje).

Fuente: elaboración propia.					
	media Z	media X	media Y	paridad	corr. vec.
<i>Coeficiente de determinación sobre el factor de supervivencia</i>					
No mitigar	-0,17	-0,23	-0,30	-0,82	-0,20
Factor constante	-0,48	-1,02	-0,16	-0,40	-1,26
Ridge · 1 variable	0,00	0,09	0,09	0,72	0,42
Ridge · 34 variables	0,15	0,41	0,39	0,84	0,29
Random Forest	0,42	0,45	0,42	0,90	0,55
GNN · SAGE	0,36	0,50	0,42	0,81	0,61
GNN · GIN	0,37	0,48	0,41	0,81	0,62
GNN · GATv2	0,36	0,49	0,40	0,83	0,60
<i>Mejora relativa sobre no mitigar</i>					
No mitigar	0,00	0,00	0,00	0,00	0,00
Factor constante	-4,77	-1,00	-2,75	-0,76	-5,63
Ridge · 1 variable	-1,82	0,22	-4,05	-2,72	-2,00
Ridge · 34 variables	-1,99	0,50	-1,49	-1,91	-4,19
Random Forest	0,33	0,81	0,37	0,73	0,01
GNN · SAGE	0,14	0,73	0,10	0,12	-0,05
GNN · GIN	-0,02	0,79	0,28	0,44	0,19
GNN · GATv2	0,06	0,76	0,22	-0,29	-0,46

Conjunto de prueba al completo (4000 circuitos), con el mismo filtro que el resto del capítulo. En negrita, el mejor de cada columna antes de redondear: en la media de Y los dos primeros redondean a 0,42 y solo uno está en negrita.

7. Conclusiones y trabajo futuro

7.1. Conclusiones

Cumplimiento de los objetivos

Los cinco objetivos específicos del apartado 3.2 se cierran así:

1. **Análisis del estado del arte y contextualización del problema.** Este objetivo quedó cubierto en el Capítulo 2.2. La revisión realizada permitió identificar con claridad el espacio que pretende abordar este trabajo. Las técnicas de mitigación más utilizadas actúan *después* de la medición y, por lo general, es necesario ejecutar el circuito un gran número de veces para corregir los efectos del ruido. Como consecuencia, el coste recae sobre el recurso más limitado en la computación cuántica actual: el tiempo de ejecución y la disponibilidad de mediciones. Sin embargo, en esta memoria se ha investigado la posibilidad de abordar el problema desde una perspectiva complementaria, es decir, anticipar la degradación *antes* de consumir esos recursos. Sobre esta idea se construye la comparativa desarrollada en este trabajo.
2. **Construcción del conjunto de datos.** Cumplido. Veinte mil circuitos de cinco a quince qubits, etiquetados con su degradación exacta sobre la calibración real de un procesador de IBM a lo largo de cuarenta y dos días (Capítulo 4). El conjunto de prueba está construido para obligar a generalizar en tres ejes independientes: tipo, tamaño y tiempo.
3. **Estrategias de representación de la información.** Cumplido, y con un hallazgo que reorientó el trabajo. La degradación con signo **no es predecible antes de ejecutar**. Su reformulación como **factor de supervivencia** sí es aprendible (apartado 4.1.2), y sobre ella se sostiene todo lo demás.
4. **Entrenar y validar los modelos con un protocolo homogéneo.** Cumplido: los tres comparten partición, umbral de señal y métricas, y la diferencia entre ellos es únicamente el modelo. Sobre el modelo óptimo que el objetivo pedía determinar, el resultado obtenido: **no existe uno estrictamente mejor**. Random Forest gana en validación en los cinco observables (entre 0,42 y 0,67 de coeficiente de determinación) y es **el único que nunca empeora** el observable al corregirlo, de +0,01 a +0,81

de mejora relativa. Pero gana por centésimas, y en el eje temporal ningún modelo se distingue de los demás: los cinco suben y bajan juntos a lo largo de los doce días (apartado 6.1.1).

5. **Comparar el rendimiento con métricas objetivas.** Cumplido, y la comparación deja dos respuestas que no coinciden.

La primera es **cuánto aporta la estructura del circuito**, que era la pregunta principal del trabajo, y la respuesta es en gran parte negativa: la representación en grafo solo gana **al extrapolar en número de qubits**, donde saca a Random Forest 0,21 y 0,23 puntos de coeficiente sobre la media de X a catorce y quince qubits (apartado 6.1.2). En el régimen conocido, treinta y cuatro columnas agregadas bastan, y cuestan muchos menos parámetros.

La segunda es que **acertar sobre el factor no garantiza mitigar bien**. Ridge tiene coeficiente positivo en los cinco observables y aun así, aplicado, empeora el resultado en cuatro de ellos.

El **objetivo general** (apartado 3.1) se cumple: las tres técnicas resuelven el mismo problema bajo un protocolo idéntico y la comparativa se resuelve sobre métricas estándar, desglosadas por observable y por eje. La **hipótesis nula** se refuta en su segunda mitad (los cinco modelos batan con margen la línea base del factor constante, que se queda en un coeficiente de cero) y solo en parte en la primera: hay diferencias medibles entre modelos, pero ninguno domina los tres ejes a la vez.

Aportaciones

1. **Un conjunto de datos que no existía.** El valor está en los tres ejes que permiten medir por separado tres formas distintas de fallar fuera de distribución. Se mezclan estos tres ejes al final para evaluar el rendimiento de los modelos con el conjunto de prueba en su totalidad.
2. **La demostración medida de que el objetivo natural del problema no es predecible antes de ejecutar**, con su explicación y su posterior reformulación.
3. **Una comparativa que separa dos preguntas:** cuánto acierta un modelo sobre el factor, y cuánto mejora el observable al aplicarlo. La primera no implica la segunda. El caso de Ridge lo demuestra.

Limitaciones

- El modelo de ruido **no incluye crosstalk**. Las conclusiones son válidas dentro de ese modelo, y un procesador donde el acoplamiento entre qubits vecinos pese más podría alterarlas.
- No hay validación en **hardware real**: el plan abierto de IBM da 10 minutos de QPU al mes (IBM Quantum, 2026b), insuficiente para etiquetar un conjunto de este tamaño. Todo lo medido procede del gemelo digital alimentado con calibración real.
- El *ground truth* exige simulación exacta, lo que **acota el tamaño** de los circuitos a quince qubits. El eje de tamaño llega hasta donde llega la memoria de la máquina, no hasta donde sería interesante.
- El transpilador de Qiskit **no es determinista**, así que la reproducibilidad del conjunto es del plan de generación.
- El trabajo solo cubre los observables cuyo **valor exacto supera el umbral de señal** de 0,002 (apartado 4.1.3). Por debajo de esa cota el factor de supervivencia es un cociente que se dispara, así que esas casillas se quedan sin etiqueta y el modelo no predice nada sobre ellas: queda fuera el 6,9 % de las etiquetas. **Los observables de señal muy pequeña están, por tanto, fuera del alcance de este trabajo.**
- El ruido se modela como un factor multiplicativo puro, **sin término afín** (apartado 4.1.3). Es una simplificación deliberada, y es la que explica los factores de supervivencia negativos que el modelo no puede representar.

7.2. Líneas de trabajo futuro

Cada una de las cinco responde a una de las limitaciones del apartado anterior. La primera es la que decide si lo medido aquí sirve fuera del simulador, y las otras cuatro amplían el alcance dentro de él.

- **Validación en hardware real.** Mientras no se haga, lo medido vale dentro del modelo de ruido y no fuera de él.
- **Incorporar el crosstalk** al modelo de ruido y volver a medir los tres ejes. Interesa especialmente el eje de tamaño, donde más qubits significan más vecinos acoplados

y donde la degradación debería notarse antes.

- **Modelar el término afín** en lugar de descartarlo, para cubrir los circuitos en los que el factor cambia de signo, que hoy quedan fuera de lo que el modelo puede representar.
- **Ampliar la batería de observables**, hoy limitada a cinco.
- **Buscar una formulación del objetivo que no sea un cociente**, para recuperar las casillas de señal pequeña que hoy retira el umbral.

Referencias

- Aktar, S., Bärtschi, A., Oyen, D., Eidenbenz, S., y Badawy, A.-H. A. (2024). Graph neural networks for parameterized quantum circuits expressibility estimation. En *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)* (pp. 1547–1557). doi: [10.1109/QCE60285.2024.00181](https://doi.org/10.1109/QCE60285.2024.00181)
- Anthropic. (2026). *Claude code (opus 5)*. <https://claude.ai/code>. ([Modelo de lenguaje de gran tamaño])
- Bao, Y., y cols. (2025). Beyond circuit connections: A non-message passing graph transformer approach for quantum error mitigation. En *International conference on learning representations (ICLR)*.
- Bao, Y., Zhong, Y., Ye, K., Tang, X., y Yan, J. (2025). QEM-Bench: Benchmarking learning-based quantum error mitigation and QEMFormer as a multi-ranged context learning baseline. En *Proceedings of the 42nd international conference on machine learning (ICML)* (Vol. 267, pp. 2953–2967).
- Biewald, L. (2020). *Experiment tracking with Weights and Biases*. Descargado de <https://www.wandb.com/> (Software disponible en wandb.com)
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. doi: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324)
- Brody, S., Alon, U., y Yahav, E. (2022). How attentive are graph attention networks? En *International conference on learning representations (ICLR)*. doi: [10.48550/arXiv.2105.14491](https://doi.org/10.48550/arXiv.2105.14491)
- Cai, Z., Babbush, R., Benjamin, S. C., Endo, S., Huggins, W. J., Li, Y., . . . O’Brien, T. E. (2023). Quantum error mitigation. *Reviews of Modern Physics*, 95(4), 045005. doi: [10.1103/RevModPhys.95.045005](https://doi.org/10.1103/RevModPhys.95.045005)
- Fey, M., y Lenssen, J. E. (2019). *Fast graph representation learning with PyTorch Geometric*. (Preprint) doi: [10.48550/arXiv.1903.02428](https://doi.org/10.48550/arXiv.1903.02428)
- Google. (2026a). *Gemini pro (modo Deep Research)*. <https://gemini.google.com>. ([Modelo de lenguaje de gran tamaño])
- Google. (2026b). *Google cloud compute engine*. <https://cloud.google.com/compute>. (Consultado en septiembre de 2026)
- Hamilton, W. L., Ying, R., y Leskovec, J. (2017). Inductive representation learning on large graphs. En *Advances in neural information processing systems (NeurIPS)*. doi:

[10.48550/arXiv.1706.02216](https://arxiv.org/abs/1706.02216)

- Hartnett, G. S., Barbosa, A., Mundada, P. S., Hush, M. J., y cols. (2024). Learning to rank quantum circuits for hardware-optimized performance enhancement. *Quantum*, 8, 1542. doi: [10.22331/q-2024-11-14-1542](https://doi.org/10.22331/q-2024-11-14-1542)
- Hirasaki, Y., Daimon, S., Itoko, T., Kanazawa, N., y Saitoh, E. (2023). Detection of temporal fluctuation in superconducting qubits for quantum error mitigation. *Applied Physics Letters*, 123(18), 184002. doi: [10.1063/5.0166739](https://doi.org/10.1063/5.0166739)
- Hoerl, A. E., y Kennard, R. W. (1970). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67. doi: [10.1080/00401706.1970.10488634](https://doi.org/10.1080/00401706.1970.10488634)
- Huo, S., Leeds, C., Ludmir, E., DiBrita, N. S., y Patel, T. (2026). Anchor: Reducing temporal and spatial output performance variability on quantum computers. En *Proceedings of the acm on measurement and analysis of computing systems (sigmetrics)*.
- IBM Quantum. (2026a). *Hardware for useful quantum computing*. <https://www.ibm.com/quantum/hardware>. (Consultado el 9 de septiembre de 2026)
- IBM Quantum. (2026b). *IBM quantum computing: products and services*. <https://www.ibm.com/quantum/products>. (Consultado el 11 de septiembre de 2026)
- Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., ... Gambetta, J. M. (2024). *Quantum computing with Qiskit*. (Preprint) doi: [10.48550/arXiv.2405.08810](https://arxiv.org/abs/2405.08810)
- Liao, H., Wang, D. S., Sitdikov, I., Salcedo, C., Seif, A., y Mineev, Z. K. (2024). Machine learning for practical quantum error mitigation. *Nature Machine Intelligence*, 6, 1478–1486. doi: [10.1038/s42256-024-00927-2](https://doi.org/10.1038/s42256-024-00927-2)
- Liu, Y., y cols. (2026). Output prediction of quantum circuits based on graph neural networks. *Frontiers of Physics*, 21(6), 063201. doi: [10.15302/frontphys.2026.063201](https://doi.org/10.15302/frontphys.2026.063201)
- Martyniuk, D., Jung, J., Barta, D., y Paschke, A. (2025). *Benchmarking quantum architecture search with surrogate assistance*. (Preprint) doi: [10.48550/arXiv.2506.06762](https://arxiv.org/abs/2506.06762)
- Nation, P. D., y Treinish, M. (2023). Suppressing quantum circuit errors due to system variability. *PRX Quantum*, 4(1), 010327. doi: [10.1103/PRXQuantum.4.010327](https://doi.org/10.1103/PRXQuantum.4.010327)
- Nielsen, M. A., y Chuang, I. L. (2010). *Quantum computation and quantum information* (10th Anniversary ed.). Cambridge: Cambridge University Press. doi: [10.1017/CBO9780511976667](https://doi.org/10.1017/CBO9780511976667)
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S.

- (2019). PyTorch: An imperative style, high-performance deep learning library. En *Advances in neural information processing systems 32* (pp. 8024–8035).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Placidi, L., y cols. (2026). *Deep learning approaches to quantum error mitigation*. (Preprint, sin revisión por pares)
- Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, 2, 79. doi: [10.22331/q-2018-08-06-79](https://doi.org/10.22331/q-2018-08-06-79)
- Schneider, J., y Smalley, I. (2026). *¿qué es la computación cuántica?* IBM Think. <https://www.ibm.com/es-es/think/topics/quantum-computing>. (Consultado el 9 de septiembre de 2026)
- Schröer, C., Kruse, F., y Gómez, J. M. (2021). A systematic literature review on applying CRISP-DM process model. *Procedia Computer Science*, 181, 526–534. doi: [10.1016/j.procs.2021.01.199](https://doi.org/10.1016/j.procs.2021.01.199)
- Shor, P. W. (1994). Algorithms for quantum computation: discrete logarithms and factoring. En *Proceedings 35th annual symposium on foundations of computer science* (pp. 124–134). IEEE Computer Society Press. doi: [10.1109/SFCS.1994.365700](https://doi.org/10.1109/SFCS.1994.365700)
- Temme, K., Bravyi, S., y Gambetta, J. M. (2017). Error mitigation for short-depth quantum circuits. *Physical Review Letters*, 119(18), 180509. doi: [10.1103/PhysRevLett.119.180509](https://doi.org/10.1103/PhysRevLett.119.180509)
- Tudisco, A., Volpe, D., Orlandi, G., y Turvani, G. (2025). Graph neural network-based predictor for optimal quantum hardware selection. En *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)* (pp. 296–301). doi: [10.1109/QCE65121.2025.10339](https://doi.org/10.1109/QCE65121.2025.10339)
- Ur Rehman, M. (2026). *What is NISQ quantum computing: The current era of quantum machines*. The Quantum Insider. <https://thequantuminsider.com/2026/08/21/what-is-nisq-quantum-computing-guide/>. (Publicado el 21 de agosto de 2026. Consultado el 9 de septiembre de 2026)
- Wang, H., Liu, P., Cheng, J., Liang, Z., Gu, J., Li, Z., ... Han, S. (2025). *QuEst: Graph transformer for quantum circuit reliability estimation*. (Preprint) doi: [10.48550/arXiv.2210.16724](https://doi.org/10.48550/arXiv.2210.16724)
- Wirth, R., y Hipp, J. (2000). CRISP-DM: Towards a standard process model for da-

ta mining. En *Proceedings of the fourth international conference on the practical application of knowledge discovery and data mining* (pp. 29–39).

Xu, K., Hu, W., Leskovec, J., y Jegelka, S. (2019). How powerful are graph neural networks? En *International conference on learning representations (iclr)*. doi: [10.48550/arXiv.1810.00826](https://doi.org/10.48550/arXiv.1810.00826)

A. Herramientas empleadas

La generación del conjunto de datos y la simulación del ruido se realizan con **Qiskit** y su simulador Aer (Javadi-Abhari y cols., 2024). Los modelos tabulares se implementan con **scikit-learn** (Pedregosa y cols., 2011), y la red neuronal de grafos con **PyTorch** (Paszke y cols., 2019) y **PyTorch Geometric** (Fey y Lenssen, 2019). El seguimiento de los experimentos, con su semilla, la versión del código y la huella del conjunto de datos, se lleva con **Weights & Biases** (Biewald, 2020).

El cómputo se reparte entre la máquina de desarrollo y **Google Cloud** (Google, 2026b). Esta última es una necesidad: las muestras de mayor tamaño exigen más memoria de la que tiene la máquina local, y el entrenamiento de la red de grafos requiere una unidad de procesamiento gráfico.

B. Declaración de uso de herramientas de inteligencia artificial

B.1. Alcance de la declaración

Este anexo detalla, de forma exhaustiva, en qué tareas se han empleado herramientas de inteligencia artificial generativa durante la realización del trabajo, en cumplimiento de lo establecido en la guía de la asignatura.

B.2. Herramientas empleadas y uso concreto

Herramienta	Uso
Claude Code (Anthropic)	Desarrollo del código de generación y simulación de circuitos cuánticos con Qiskit, y del resto de código ajeno al aprendizaje automático.
Claude Code (Anthropic)	Generación de las figuras e ilustraciones de la memoria.
Claude Code (Anthropic)	Asistencia en la redacción, la estructuración y la revisión del documento.
Google Gemini	Búsqueda bibliográfica en profundidad para la elaboración del estado del arte.

Tabla B.1: Herramientas de inteligencia artificial empleadas y tareas en las que se han utilizado.

Las dos herramientas quedan referenciadas según la normativa APA que exige la titulación: **Claude Code** (Anthropic, 2026) y **Gemini Pro** (Google, 2026a).

Las figuras de la memoria son de **dos clases distintas**, y su pie lo refleja. Las gráficas que representan medidas del trabajo se han producido con código propio sobre los datos generados para esta memoria, y su fuente es elaboración propia; los **diagramas conceptuales**, que no representan ningún dato, se han elaborado con Claude Code y llevan esa herramienta como fuente.